Department of Computer Science & Engineering—Cyber Security & Data Science

Laboratory Manual

Bachelor of Technology in Computer Science & Engineering—Data Science
PCC-CSD691, Machine Learning and its Application

Compiled by
Mr. Susmit Chakraborty
Assistant Professor
Dept. of CSE- CS & DS
Brainware University

BRAINWARE UNIVERSITY
School of Engineering
Department of Computer Science & Engineering—Cyber Security & Data Science
398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

Table of Contents

Date	Compiled by	Description
January 2025	Mr. Susmit Chakraborty	Updated with 14 points for each example
January 2023	Dept. of CSE	Initial release

Table: Revision History

BRAINWARE UNIVERSITY
School of Engineering
Department of Computer Science & Engineering—Cyber Security & Data Science
398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

Experiment No 1 (To perform different Arithmetic Operations on numbers in Python)

Aim/Purpose of the Experiment

To familiarize the students with different algebraic operations in python environment.

Learning Outcomes
Knowledge of the mathematical operations with user defined datasets in python.

Prerequisites

Basic knowledge of programming. Basic algebra.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory

Basic algebraic operations are fundamental mathematical operations that involve manipulating numbers and symbols according to certain rules. These operations are essential for solving equations, simplifying expressions, and understanding mathematical relationships.
Addition (+): Adds two numbers.
Subtraction (-): Subtracts the second number from the first.
Multiplication (*): Multiplies two numbers.
Division (/): Divides the first number by the second (results in a floating-point number).
Integer Division (//): Divides the first number by the second and returns an integer.
Modulo (%): Returns the remainder of the division of the first number by the second.
Exponentiation (**): Raises the first number to the power of the second.

```
# Addition
result_addition = num1 + num2
print("Addition:", result_addition)

# Subtraction
result_subtraction = num1 - num2

# Division
result_division = num1 / num2
print("Division:", result_division)

# Integer Division
result_integer_division = num1 // num2
print("Integer Division:", result_integer_division)

# Modulo
result_modulo = num1 % num2
```

Operating Procedure

```
print("Subtraction:", result_subtraction)

# Multiplication
result_multiplication = num1 * num2
print("Multiplication:", result_multiplication)


print("Modulo:", result_modulo)

# Exponentiation
result_exponentiation = num1 ** num2
print("Exponentiation:", result_exponentiation)


# Taking inputs from the user
num1 = int(input("Enter your name: "))
num2 = int(input("Enter your age: "))
```

Open Jupyter note book
Take a new python file
Type the code
Run it
Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting

Precautions:

Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss.
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable..
Readability: Write clean, well-commented code; use markdown for explanations.
Check Dependencies: Ensure all required libraries are installed using !pip list or !conda list..
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.

Troubleshooting:

Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs..
Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.
Browser Issues: Clear browser cache or switch browsers if the UI misbehaves..
Documentation: Refer to Jupyter's official docs and community forums for help.

Observations

Observe the results obtained in each operation.

Calculations & Analysis

Calculations should be given for each operation.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions
What is Python?
What is the difference between Python Arrays and lists?
What are the common built-in data types in Python?
What are local variables and global variables in Python?
What is type conversion in Python?
Explain the requirement of indentation in python.
What is the method to write comments in Python?
Is Python fully object oriented?
What is the difference between %,/,// ?

Extension and Follow-up Activities (if applicable)
NA
Assessments
Suggested reading
NA

Experiment No 2 (Write a program to compute summary statistics such as mean, median, mode, standard deviation and variance of the given different types of data)

Aim/Purpose of the Experiment

To familiarize the students with different statistical operations in python environment.

Learning Outcomes
Knowledge of the statistical operations with user defined datasets in python.

Prerequisites

Basic knowledge of programming. Basic algebra.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory

Statistics is a branch of mathematics that deals with collecting, analyzing, interpreting, presenting, and organizing data. Basic statistics provide essential tools for summarizing and understanding data, making informed decisions, and drawing conclusions from observations or experiments.
Mean: The arithmetic mean, often simply called the "mean," is the sum of all values in a dataset divided by the number of values. It represents the average value of the data.

Median: The median is the middle value of a dataset when the values are sorted in ascending or descending order. If the dataset has an even number of values, the median is the average of the two middle values.

Mode: The mode is the value that appears most frequently in a dataset. A dataset can have one mode (unimodal), two modes (bimodal), or more modes (multimodal).

Variance: The variance measures how much the values in a dataset deviate from the mean. It is the average of the squared differences between each value and the mean.

Standard Deviation: The standard deviation is the square root of the variance. It provides a measure of the average distance between each data point and the mean.

import statistics

```
def compute_summary_statistics(data):
    mean = statistics.mean(data)
    median = statistics.median(data)
    try:
        mode = statistics.mode(data)
    except statistics.StatisticsError:
        mode = "No unique mode found"
    std_dev = statistics.stdev(data)
    variance = statistics.variance(data)

    return mean, median, mode, std_dev, variance

# User Defined dataset
n = int(input("enter the counts of the numbers: "))
numbers = []
for i in range(n):
    a = int(input("enter the number: "))
    numbers.append(a)
print(numbers)

# Compute summary statistics
mean, median, mode, std_dev, variance = compute_summary_statistics(numbers)

# Print results
print("Mean:", mean)
print("Median:", median)
print("Mode:", mode)
print("Standard Deviation:", std_dev)
print("Variance:", variance)
```

Operating Procedure

Open Jupyter note book
Take a new python file
Type the code
Run it
Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting

Precautions:

Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss.
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable..
Readability: Write clean, well-commented code; use markdown for explanations.
Check Dependencies: Ensure all required libraries are installed using !pip list or !conda list..
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.

Troubleshooting:

Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs..
Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.
Browser Issues: Clear browser cache or switch browsers if the UI misbehaves.
Documentation: Refer to Jupyter's official docs and community forums for help.

Observations

Observe the results obtained in each operation.

Calculations & Analysis

Calculations should be given for each operation.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions

What is Python?
What is the difference between Python Arrays and lists?
What are the common built-in data types in Python?
What are local variables and global variables in Python?
What is type conversion in Python?
Explain the requirement of indentation in python.
What is the method to write comments in Python?
Is Python fully object oriented?
What is package?
What is the try method in python?
What is the except method in python?
What is function in python?

Extension and Follow-up Activities (if applicable)

NA

Assessments

Suggested reading

NA

Experiment No 3 (Plot the different results obtained by algebraic and statistical operations in python)

Aim/Purpose of the Experiment

To familiarize the students with different graphical representation methods in python environment.

Learning Outcomes
Knowledge of different graphical representation methods in python.

Prerequisites

Basic knowledge of programming. Basic algebra. Python Packages.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory

Data visualization is an important skill to possess for anyone trying to extract and communicate insights from data. Great business narratives and presentations often stem from brilliant visualizations that convey the key ideas in a concise and aesthetic manner. In the field of machine learning, visualization plays a key role throughout the entire process of analysis - to obtain relationships, observe trends and portray the final results as well.
There are two basic libraries that deals with the data visualization.
matplotlib.pyplot
seaborn
The Data visualization methods are as follows:
Bar chart
Scatter plot
Line graph
Histogram
Box plot

Example - 1
Bar Chart - Plot sales across each product category
A bar chart uses bars to show comparisons between categories of data.
A bar graph will always have two axis.
One axis will generally have numerical values or measures,
The other will describe the types of categories being compared or dimensions.

```
import numpy as np
import matplotlib.pyplot as plt
product_category = np.array(['Furniture', 'Technology', 'Office Supplies'])
sales = np.array ([4110451.90, 4744557.50, 3787492.52] )
plt.bar(product_category, sales)
plt.show()
```

Bar Chart - Plot sales across each product category
Adding labels to Axes
Reducing the bar width
Giving Title to the chart
Modifying the ticks to show information in (million dollars)

```
import numpy as np
import matplotlib.pyplot as plt
product_category = np.array(['Furniture', 'Technology', 'Office Supplies'])
sales = np.array ([4110451.90, 4744557.50, 3787492.52] )
# plotting bar chart and setting bar width to 0.5 and aligning it to center
plt.bar(product_category, sales, width= 0.5, align='center', edgecolor='Orange',color='cyan')
# Adding and formatting title
plt.title("Sales Across Product Categories\n", fontdict={'fontsize': 20, 'fontweight' : 5, 'color' : 'Green'})
# Labeling Axes
plt.xlabel("Product Category", fontdict={'fontsize': 12, 'fontweight' : 5, 'color' : 'Brown'})
plt.ylabel("Sales", fontdict={'fontsize': 12, 'fontweight' : 5, 'color' : 'Brown'})

# Modifying the ticks to show information in (million dollars)
ticks = np.arange(0, 6000000, 1000000)
labels = ["{}M".format(i//1000000) for i in ticks]
plt.yticks(ticks, labels)
plt.show()
```

Example - 2
Scatter Chart - Plot Sales versus Profits across various Countries and Product Categories
Scatter plots are used when you want to show the relationship between two facts or measures.
Profit is measured across each product category

```
import numpy as np
import matplotlib.pyplot as plt
# Data
sales = np.array ([1013.14, 8298.48, 875.51,.....89])
product_category = np.array(['Technology', 'Technology'..... 'Furniture', 'Furniture'])
country = np.array(['Zimbabwe', 'Zambia', 'Yemen', ...., 'Algeria', 'Albania', 'Afghanistan'])

# plotting scatter chart
```

```
plt.scatter(profit, sales, alpha= 0.7, s = 50 )

# Adding and formatting title
plt.title("Sales versus Profits across various Countries and Product Categories\n", fontdict={'fontsize': 20,
'fontweight' : 5, 'color' : 'Green'})

# Labeling Axes
plt.xlabel("Profit", fontdict={'fontsize': 12, 'fontweight' : 5, 'color' : 'Brown'})
plt.ylabel("Sales", fontdict={'fontsize': 12, 'fontweight' : 5, 'color' : 'Brown'})

plt.show()
```

Scatter Chart - Plot Sales versus Profits across various Countries and Product Categories
Represent product category using different colors
Adding a Legend to Product Categories

```
plt.scatter(profit[product_category == "Technology"], sales[product_category == "Technology"],
        c= 'Green', alpha= 0.7, s = 150, label="Technology" )

plt.scatter(profit[product_category == "Office Supplies"], sales[product_category == "Office Supplies"],
        c= 'Yellow', alpha= 0.7, s = 100, label="Office Supplies" )

plt.scatter(profit[product_category == "Furniture"], sales[product_category == "Furniture"],
        c= 'Cyan', alpha= 0.7, s = 50, label="Furniture" )

# Adding and formatting title
plt.title("Sales versus Profits across various Countries and Product Categories\n", fontdict={'fontsize': 20,
'fontweight' : 5, 'color' : 'Green'})

# Labeling Axes
plt.xlabel("Profit", fontdict={'fontsize': 12, 'fontweight' : 5, 'color' : 'Brown'})
plt.ylabel("Sales", fontdict={'fontsize': 12, 'fontweight' : 5, 'color' : 'Brown'}
plt.legend()
plt.show()
```

Example – 3
Line Chart - Plot Sales across 2015

```
import numpy as np
import matplotlib.pyplot as plt

months = np.array(['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October',
'November', 'December'])
sales = np.array([241268.56, 184837.36, 263100.77, 242771.86, 288401.05, 401814.06, 258705.68, 456619.94,
481157.24, 422766.63, 555279.03, 503143.69])
plt.plot(months, sales)

# Adding and formatting title
```

```
plt.title("Sales across 2015\n", fontdict={'fontsize': 20, 'fontweight' : 5, 'color' : 'Green'})

# Labeling Axes
plt.xlabel("Months", fontdict={'fontsize': 12, 'fontweight' : 5, 'color' : 'Brown'})
plt.ylabel("Sales", fontdict={'fontsize': 12, 'fontweight' : 5, 'color' : 'Brown'} )

ticks = np.arange(0, 600000, 50000)
labels = ["{}K".format(i//1000) for i in ticks]
plt.yticks(ticks, labels)
plt.xticks(rotation=90)
plt.show()
plt.plot(months, sales)

# Adding and formatting title
plt.title("Sales across 2015\n", fontdict={'fontsize': 20, 'fontweight' : 5, 'color' : 'Green'})

# Labeling Axes
plt.xlabel("Months", fontdict={'fontsize': 12, 'fontweight' : 5, 'color' : 'Brown'})
plt.ylabel("Sales", fontdict={'fontsize': 12, 'fontweight' : 5, 'color' : 'Brown'} )

ticks = np.arange(0, 600000, 50000)
labels = ["{}K".format(i//1000) for i in ticks]
plt.yticks(ticks, labels)

plt.xticks(rotation=90)

for xy in zip(months, sales):
   plt.annotate(s = "{}K".format(xy[1]//1000), xy = xy,  textcoords='data')

plt.show()
```

Example - 4
Box and Whisker Chart - Sales across Countries and Product Categories
A Box and Whisker Plot (or Box Plot) is a convenient way of visually displaying the data distribution through their quartiles. The lines extending parallel from the boxes are known as the “whiskers”, which are used to indicate variability outside the upper and lower quartiles. Outliers are sometimes plotted as individual dots that are in-line with whiskers. Box Plots can be drawn either vertically or horizontally.

```
mport numpy as np
import matplotlib.pyplot as plt

# Data
sales_technology = np.array ([1013.14…., 6462.57])
sales_office_supplies = np.array ([1770.13, 7527.18, ……7, 611.82, 3925.56])
sales_furniture = np.array ([981.84,.58, 24699.51,…… 23524.51, 8731.68, 8425.86, 835.95, 11285.19])
```

```
plt.boxplot([sales_technology, sales_office_supplies, sales_furniture])
# Adding and formatting title
plt.title("Sales across Countries and Product Categories\n", fontdict={'fontsize': 20, 'fontweight' : 5, 'color' : 'Green'})
# Labeling Axes
plt.xlabel("Product Category", fontdict={'fontsize': 12, 'fontweight' : 5, 'color' : 'Brown'})
plt.ylabel("Sales", fontdict={'fontsize': 12, 'fontweight' : 5, 'color' : 'Brown'})
plt.xticks((1,2,3),["Technology", "Office Supplies", "Furniture"])
plt.sho
```

Example - 5

Histogram - Plot a histogram for Sales distribution across Countries.

A histogram is a plot that lets you discover, and show, the underlying frequency distribution (shape) of a set of continuous data.

```
import numpy as np
import matplotlib.pyplot as plt
# Data
profit = np.array([-5428.79, 7001.7572.59,831.05, 24341.7, ……. 06.5, 709.32, 5460.3])
plt.hist(profit, bins = 100,edgecolor='Orange',color='cyan')
plt.show()
```

Operating Procedure

Open Jupyter note book
Take a new python file
Type the code
Run it
Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting

Precautions:

Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss.
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable..
Readability: Write clean, well-commented code; use markdown for explanations.
Check Dependencies: Ensure all required libraries are installed using !pip list or !conda list.
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.

Troubleshooting:

Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs..
Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.

Browser Issues: Clear browser cache or switch browsers if the UI misbehave.
Documentation: Refer to Jupyter's official docs and community forums for help.

Observations
Observe the results obtained in each operation.

Calculations & Analysis
Calculations should be given for each operation.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions
How can you create a histogram in Matplotlib?
How can you add a legend to a plot in Matplotlib?
How can you set the font size of a plot in Matplotlib?
What is the difference between a scatter plot and a line plot in Matplotlib?
How can you add text to a plot in Matplotlib?
What is the difference between a bar plot and a histogram in Matplotlib?
How can you set the color of a plot in Matplotlib?
What is the purpose of the plt.grid() function in Matplotlib?
How can you create a box plot in Matplotlib?
How can you set the size of a plot in Matplotlib?
How can you change the linestyle of a plot in Matplotlib?
How can you change the marker style of a plot in Matplotlib?
Extension and Follow-up Activities (if applicable)
NA
Assessments
Suggested reading
NA

Experiment No 4 (Model building using Linear Regression)

Aim/Purpose of the Experiment
Model building using Linear Regression.

Learning Outcomes
Knowledge of the Data cleaning, modelling using linear regression, and different libraries in python.

Prerequisites

Basic knowledge of programming. Coordinate geometry (Straight line), Matplotlib, seaborn, etc.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory

Linear regression is a fundamental statistical method used for modeling the relationship between a dependent variable and one or more independent variables. It's often employed in predictive analysis and understanding the association between variables.

Dependent variable (Y): This is the variable that you want to predict or explain. It's also known as the response variable.

Independent variable(s) (X): These are the variables that are used to predict the dependent variable. They are also referred to as predictor variables or features.

Linear relationship: Linear regression assumes that there is a linear relationship between the independent variables and the dependent variable. This means that the change in the dependent variable is directly proportional to a change in the independent variable(s), with a constant rate of change.

Simple linear regression: When there is only one independent variable, it's called simple linear regression. The relationship between the independent and dependent variables is modeled using a straight line equation: $Y = \beta_o + \beta_1 X + \varepsilon$, where β_o is the intercept, β_1 is the slope coefficient, X is the independent variable, and ε represents the error term.

Multiple linear regression: When there are multiple independent variables, it's called multiple linear regression. The relationship between the independent and dependent variables is modeled using the equation: $Y = \beta_o + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_r X_r + \varepsilon$, where $X_1, X_2, ..., X_r$ are the independent variables, β_o is the intercept, $\beta_1, \beta_2, ..., \beta_r$ are the coefficients, and ε represents the error term.
Simple Linear Regression
Step 1: Reading and Understanding the Data
Let's start with the following steps:
Importing data using the pandas library

Understanding the structure of the data

```
#Supress Warnings
import warnings
warnings.filterwarnings('ignore')
# Import the numpy and pandas package
import numpy as np
import pandas as pd
# Read the given CSV file, and view some sample records
advertising = pd.read_csv("advertising.csv")
advertising.head()
advertising.shape
advertising.info()
advertising.describe()
```

Step 2: Visualising the Data

Let's now visualise our data using seaborn. We'll first make a pairplot of all the variables present to visualise which variables are most correlated to Sales

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.pairplot(advertising, x_vars=['TV', 'Newspaper', 'Radio'], y_vars='Sales',size=4, aspect=1, kind='scatter')
plt.show()
sns.heatmap(advertising.corr(), cmap="YlGnBu", annot = True)
plt.show()
```

Step 3: Performing Simple Linear Regression

Equation of linear regression

$y = c + m_1x_1 + m_2x_2 + \ldots + m_nx_n$

- y is the response
- c is the intercept
- m_1 is the coefficient for the first feature
- m_n is the coefficient for the nth feature

In our case:

$y = c + m_1 \times TV$

The m values are called the model **coefficients** or **model parameters**.

Generic Steps in model building using statsmodels

We first assign the feature variable, TV, in this case, to the variable X and the response variable, Sales, to the variable y.

```
X = advertising['TV']
y = advertising['Sales']
```

Train-Test Split

You now need to split our variable into training and testing sets. You'll perform this by importing train_test_split from the sklearn.model_selection library. It is usually a good practice to keep 70% of the data in your train dataset

and the rest 30% in your test dataset

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.7, test_size = 0.3, random_state = 100)
# Let's now take a look at the train dataset
X_train.head()
y_train.head()
```

Building a Linear Model

You first need to import the statsmodel.api library using which you'll perform the linear regression.

```
import statsmodels.api as sm
# Add a constant to get an intercept
X_train_sm = sm.add_constant(X_train)
# Fit the resgression line using 'OLS'
lr = sm.OLS(y_train, X_train_sm).fit()
# Print the parameters, i.e. the intercept and the slope of the regression line fitted
lr.params
# Performing a summary operation lists out all the different parameters of the regression line fitted
print(lr.summary())
```

Operating Procedure

Open Jupyter note book
Take a new python file
Type the code
Run it
Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting

Precautions:

Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss.
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable..
Readability: Write clean, well-commented code; use markdown for explanations.
Check Dependencies: Ensure all required libraries are installed using !pip list or !conda list.
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.

Troubleshooting:

Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs..
Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.
Browser Issues: Clear browser cache or switch browsers if the UI misbehaves.
Documentation: Refer to Jupyter's official docs and community forums for help.

Observations

Observe the results obtained in each step.

Calculations & Analysis
Calculations should be given for model output.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions
What is linear regression, and how does it work?
What are the assumptions of a linear regression model?
What are outliers? How do you detect and treat them? How do you deal with outliers in a linear regression model?
What is the difference between simple and multiple linear regression?
What is multicollinearity and how does it affect linear regression analysis?
What is the difference between linear regression and non-linear regression?
Can you explain the concept of overfitting in linear regression?
What are the limitations of linear regression?
What is the curse of dimensionality?
Extension and Follow-up Activities (if applicable)
NA
Assessments
Suggested reading
NA

Experiment No 5 (To familiarize the students with Model building using Logistic Regression)

Aim/Purpose of the Experiment

To familiarize the students with Model building using Logistic Regression.

Learning Outcomes
Knowledge of the Data cleaning, modelling using logistic regression, and different libraries in python.

Prerequisites

Basic knowledge of programming, python syntax, matplotlib, seaborn, different libraries.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory

Logistic regression is a statistical method used for modeling the probability of a binary outcome based on one or more predictor variables. It's widely used for classification tasks where the dependent variable is categorical and has two possible outcomes.
Here's an overview of the key concepts and components of logistic regression:

Binary outcome: Logistic regression is specifically designed for situations where the dependent variable (also known as the response variable or target variable) is binary, meaning it has only two possible outcomes. These outcomes are typically represented as 0 and 1, or as "success" and "failure", "yes" and "no", etc.
Logistic function (sigmoid function): In logistic regression, the relationship between the predictor variables and the probability of the binary outcome is modeled using the logistic function, also known as the sigmoid function. The logistic function is an S-shaped curve that maps any real-valued number to a value between 0 and 1, representing probabilities.
Probability prediction: Unlike linear regression, where the output is continuous, logistic regression predicts the probability that a given observation belongs to a particular category (e.g., the probability of a customer buying a product). The predicted probabilities are then used to make classifications.
Logit transformation: The logistic function is expressed in terms of the log-odds, also known as the logit function. The logit of the probability of the event occurring (p) is defined as the logarithm of the odds ratio (p / (1 - p)). Mathematically, it can be represented as log(p / (1 - p)).
Model parameters: Similar to linear regression, logistic regression estimates parameters (coefficients) that define the relationship between the predictor variables and the log-odds of the binary outcome. These parameters are estimated using maximum likelihood estimation or other optimization techniques.
Interpretation of coefficients: The coefficients obtained from logistic regression represent the change in the log-odds of the outcome associated with a one-unit change in the corresponding predictor variable, holding other variables constant.
Decision boundary: In logistic regression, a decision boundary is used to classify observations into different categories based on their predicted probabilities. The decision boundary is typically set at 0.5, meaning that

observations with predicted probabilities greater than 0.5 are classified into one category, while those with predicted probabilities less than or equal to 0.5 are classified into the other category.

Logistic Regression
Step 1: Importing and Merging Data

```
# Suppressing Warnings
import warnings
warnings.filterwarning
# Importing Pandas and NumPy
import pandas as pd, numpy as np
# Importing all datasets
churn_data = pd.read_csv("churn_data.csv")
churn_data.head()
customer_data = pd.read_csv("customer_data.csv")
customer_data.head()
internet_data = pd.read_csv("internet_data.csv")
internet_data.head()

#Combining all data files into one consolidated dataframe
# Merging on 'customerID'
df_1 = pd.merge(churn_data, customer_data, how='inner', on='customerID')
# Final dataframe with all predictor variables
telecom = pd.merge(df_1, internet_data, how='inner', on='customerID')
```

Step 2: Inspecting the Dataframe

```
# Let's see the head of our master dataset
telecom.head()
# Let's check the dimensions of the dataframe
telecom.shape
# let's look at the statistical aspects of the dataframe
telecom.describe()
# Let's see the type of each column
telecom.info()
```

Step 3: Data Preparation

```
#Converting some binary variables (Yes/No) to 0/1
# List of variables to map
varlist =  ['PhoneService', 'PaperlessBilling', 'Churn', 'Partner', 'Dependents']
# Defining the map function
def binary_map(x):
   return x.map({'Yes': 1, "No": 0})
# Applying the function to the housing list
telecom[varlist] = telecom[varlist].apply(binary_map)
telecom.head()
#For categorical variables with multiple levels, create dummy features (one-hot encoded)
# Creating a dummy variable for some of the categorical variables and dropping the first one.
dummy1 = pd.get_dummies(telecom[['Contract', 'PaymentMethod', 'gender', 'InternetService']], drop_first=True)
# Adding the results to the master dataframe
```

```
telecom = pd.concat([telecom, dummy1], axi
telecom.head()
# Creating dummy variables for the remaining categorical variables and dropping the level with big names.
# Creating dummy variables for the variable 'MultipleLines'
ml = pd.get_dummies(telecom['MultipleLines'], prefix='MultipleLines')
# Dropping MultipleLines_No phone service column
ml1 = ml.drop(['MultipleLines_No phone service'], 1)
#Adding the results to the master dataframe
telecom = pd.concat([telecom,ml1], axi
# Creating dummy variables for the variable 'OnlineSecurity'.
os = pd.get_dummies(telecom['OnlineSecurity'], prefix='OnlineSecurity')
os1 = os.drop(['OnlineSecurity_No internet service'], 1)
# Adding the results to the master dataframe
telecom = pd.concat([telecom,os1], axis=1)
# Creating dummy variables for the variable 'OnlineBackup'.
ob = pd.get_dummies(telecom['OnlineBackup'], prefix='OnlineBackup')
ob1 = ob.drop(['OnlineBackup_No internet service'], 1)
# Adding the results to the master dataframe
telecom = pd.concat([telecom,ob1], axis=1)
# Creating dummy variables for the variable 'DeviceProtection'.
dp = pd.get_dummies(telecom['DeviceProtection'], prefix='DeviceProtection')
dp1 = dp.drop(['DeviceProtection_No internet service'], 1)
# Adding the results to the master dataframe
telecom = pd.concat([telecom,dp1], axis=1)
# Creating dummy variables for the variable 'TechSupport'.
ts = pd.get_dummies(telecom['TechSupport'], prefix='TechSupport')
ts1 = ts.drop(['TechSupport_No internet service'], 1)
# Adding the results to the master dataframe
telecom = pd.concat([telecom,ts1], axis=1)
# Creating dummy variables for the variable 'StreamingTV'.
st =pd.get_dummies(telecom['StreamingTV'], prefix='StreamingTV')
st1 = st.drop(['StreamingTV_No internet service'], 1)
# Adding the results to the master dataframe
telecom = pd.concat([telecom,st1], axis=1)
# Creating dummy variables for the variable 'StreamingMovies'.
sm = pd.get_dummies(telecom['StreamingMovies'], prefix='StreamingMovies')
sm1 = sm.drop(['StreamingMovies_No internet service'], 1)
# Adding the results to the master dataframe
telecom = pd.concat([telecom,sm1], axis=1)
telecom.head()
```

Dropping the repeated variables

```
# We have created dummies for the below variables, so we can drop them
telecom = telecom.drop(['Contract','PaymentMethod','gender','MultipleLines','InternetService', 'OnlineSecurity',
'OnlineBackup', 'DeviceProtection',
    'TechSupport', 'StreamingTV', 'StreamingMovies'], 1)
#The varaible was imported as a string we need to convert it to float
```

```
telecom['TotalCharges'] = telecom['TotalCharges'].convert_objects(convert_numeric=True)
telecom.info()
#Checking for Outliers
# Checking for outliers in the continuous variables
num_telecom = telecom[['tenure','MonthlyCharges','SeniorCitizen','TotalCharges']]
#Checking for Missing Values and Inputing Them
# Adding up the missing values (column-wise)
telecom.isnull().sum()
# Checking the percentage of missing values
round(100*(telecom.isnull().sum()/len(telecom.index)), 2)
# Removing NaN TotalCharges rows
telecom = telecom[~np.isnan(telecom['TotalCharges'])]
# Checking percentage of missing values after removing the missing values
round(100*(telecom.isnull().sum()/len(telecom.index)), 2)
Step 4: Test-Train Split
from sklearn.model_selection import train_test_split
# Putting feature variable to X
X = telecom.drop(['Churn','customerID'], axis=1)
X.head()
# Putting response variable to y
y = telecom['Churn']
y.head()
# Splitting the data into train and test
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, test_size=0.3, random_state=100)
Step 5: Feature Scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train[['tenure','MonthlyCharges','TotalCharges']] =
scaler.fit_transform(X_train[['tenure','MonthlyCharges','TotalCharges']])
X_train.head()
### Checking the Churn Rate
churn = (sum(telecom['Churn'])/len(telecom['Churn'].index))*100
churn
Step 6: Looking at Correlations
# Importing matplotlib and seaborn
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
# Let's see the correlation matrix
plt.figure(figsize = (20,10))       # Size of the figure
sns.heatmap(telecom.corr(),annot = True)
plt.show()
#Dropping highly correlated dummy variables
X_test =
X_test.drop(['MultipleLines_No','OnlineSecurity_No','OnlineBackup_No','DeviceProtection_No','TechSupport_No','St
reamingTV_No','StreamingMovies_No'], 1)
```

```
X_train =
X_train.drop(['MultipleLines_No','OnlineSecurity_No','OnlineBackup_No','DeviceProtection_No','TechSupport_No',
'StreamingTV_No','StreamingMovies_No'], 1)
#Checking the Correlation Matrix
#After dropping highly correlated variables now let's check the correlation matrix again.
plt.figure(figsize = (20,10))
sns.heatmap(X_train.corr(),annot = True)
plt.show()
Step 7: Model Building
import statsmodels.api as sm
# Logistic regression model
logm1 = sm.GLM(y_train,(sm.add_constant(X_train)), family = sm.families.Binomial())
logm1.fit().summary()
```

Operating Procedure

Open Jupyter note book
Take a new python file
Type the code
Run it
Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting

Precautions:

Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss.
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable..
Readability: Write clean, well-commented code; use markdown for explanations.
Check Dependencies: Ensure all required libraries are installed using !pip list or !conda list..
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.

Troubleshooting:

Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs..
Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.
Browser Issues: Clear browser cache or switch browsers if the UI misbehaves.
Documentation: Refer to Jupyter's official docs and community forums for help.

Observations

Observe the results obtained in each operation.

Calculations & Analysis

Calculations should be given for each operation.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions

What do you mean by the Logistic Regression?
What are the different types of Logistic Regression?
What are the odds?
What is the Impact of Outliers on Logistic Regression?
What is the difference between the outputs of the Logistic model and the Logistic function?
How do we handle categorical variables in Logistic Regression?
What are the assumptions made in Logistic Regression?
Why is Logistic Regression termed as Regression and not classification?
What are the advantages of Logistic Regression?
What are the disadvantages of Logistic Regression?
Extension and Follow-up Activities (if applicable)
NA

Assessments

Suggested reading

NA

Experiment No 6 (To familiarize the students with Model building using K-Means Clustering)

Aim/Purpose of the Experiment

To familiarize the students with Model building using K-Means Clustering.

Learning Outcomes
Knowledge of the Data cleaning, Data preparation, modelling K-Means Clustering, and different libraries in python.

Prerequisites

Basic knowledge of programming, python syntax, matplotlib, seaborn, different libraries.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory

Logistic regression is a statistical method used for modeling the probability of a binary outcome based on one or more predictor variables. It's widely used for classification tasks where the dependent variable is categorical and has two possible outcomes.
Here's an overview of the key concepts and components of logistic regression:

Binary outcome: Logistic regression is specifically designed for situations where the dependent variable (also known as the response variable or target variable) is binary, meaning it has only two possible outcomes. These outcomes are typically represented as 0 and 1, or as "success" and "failure", "yes" and "no", etc.
Logistic function (sigmoid function): In logistic regression, the relationship between the predictor variables and the probability of the binary outcome is modeled using the logistic function, also known as the sigmoid function. The logistic function is an S-shaped curve that maps any real-valued number to a value between 0 and 1, representing probabilities.
Probability prediction: Unlike linear regression, where the output is continuous, logistic regression predicts the probability that a given observation belongs to a particular category (e.g., the probability of a customer buying a product). The predicted probabilities are then used to make classifications.
Logit transformation: The logistic function is expressed in terms of the log-odds, also known as the logit function. The logit of the probability of the event occurring (p) is defined as the logarithm of the odds ratio (p / (1 - p)). Mathematically, it can be represented as log(p / (1 - p)).
Model parameters: Similar to linear regression, logistic regression estimates parameters (coefficients) that define the relationship between the predictor variables and the log-odds of the binary outcome. These parameters are estimated using maximum likelihood estimation or other optimization techniques.
Interpretation of coefficients: The coefficients obtained from logistic regression represent the change in the log-odds of the outcome associated with a one-unit change in the corresponding predictor variable, holding other variables constant.
Decision boundary: In logistic regression, a decision boundary is used to classify observations into different

categories based on their predicted probabilities. The decision boundary is typically set at 0.5, meaning that observations with predicted probabilities greater than 0.5 are classified into one category, while those with predicted probabilities less than or equal to 0.5 are classified into the other category.

K-Means Clustering

1. Read and visualise the data

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import datetime as
import sklearn
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import linkage
from scipy.cluster.hierarchy import dendrogram
from scipy.cluster.hierarchy import cut_tree

# read the dataset
retail_df = pd.read_csv("Online_Retail.csv", sep=",", encoding="ISO-8859-1", header=0)
retail_df.head()

# read the dataset
retail_df = pd.read_csv("Online_Retail.csv", sep=",", encoding="ISO-8859-1", header=0)
retail_df.head()
```

2. Clean the data

```
# missing values
round(100*(retail_df.isnull().sum())/len(retail_df), 2)

# drop all rows having missing values
retail_df = retail_df.dropna()
retail_df.shape
retail_df.head()
# new column: amount
retail_df['amount'] = retail_df['Quantity']*retail_df['UnitPrice']
retail_df.head()
```

3. Prepare the data for modelling

```
#R (Recency): Number of days since last purchase
#F (Frequency): Number of tracsactions
#M (Monetary): Total amount of transactions (revenue contributed)

# monetary
```

```
grouped_df = retail_df.groupby('CustomerID')['amount'].sum()
grouped_df = grouped_df.reset_index()
grouped_df.head()
# frequency
frequency = retail_df.groupby('CustomerID')['InvoiceNo'].count()
frequency = frequency.reset_index()
frequency.columns = ['CustomerID', 'frequency']
frequency.head()
# merge the two dfs
grouped_df = pd.merge(grouped_df, frequency, on='CustomerID', how='inner')
grouped_df.head()

retail_df.head()

# recency
# convert to datetime
retail_df['InvoiceDate'] = pd.to_datetime(retail_df['InvoiceDate'], format='%d-%m-%Y %H:%M')
retail_df.head()

# compute the max date
max_date = max(retail_df['InvoiceDate'])
max_date

# compute the diff
retail_df['diff'] = max_date - retail_df['InvoiceDate']
retail_df.head()

# recency
last_purchase = retail_df.groupby('CustomerID')['diff'].min()
last_purchase = last_purchase.reset_index()
last_purchase.head()

# merge
grouped_df = pd.merge(grouped_df, last_purchase, on='CustomerID', how='inner')
grouped_df.columns = ['CustomerID', 'amount', 'frequency', 'recency']
grouped_df.head()

# number of days only
grouped_df['recency'] = grouped_df['recency'].dt.days
grouped_df.head()

# 1. outlier treatment
plt.boxplot(grouped_df['recency'])

# two types of outliers:
# - statistical
```

```
# - domain specific
# removing (statistical) outliers
Q1 = grouped_df.amount.quantile(0.05)
Q3 = grouped_df.amount.quantile(0.95)
IQR = Q3 - Q1
grouped_df = grouped_df[(grouped_df.amount >= Q1 - 1.5*IQR) & (grouped_df.amount <= Q3 + 1.5*IQR)]
# outlier treatment for recency
Q1 = grouped_df.recency.quantile(0.05)
Q3 = grouped_df.recency.quantile(0.95)
IQR = Q3 - Q1
grouped_df = grouped_df[(grouped_df.recency >= Q1 - 1.5*IQR) & (grouped_df.recency <= Q3 + 1.5*IQR)]
# outlier treatment for frequency
Q1 = grouped_df.frequency.quantile(0.05)
Q3 = grouped_df.frequency.quantile(0.95)
IQR = Q3 - Q1
grouped_df = grouped_df[(grouped_df.frequency >= Q1 - 1.5*IQR) & (grouped_df.frequency <= Q3 + 1.5*IQR)]

# 2. rescaling
rfm_df = grouped_df[['amount', 'frequency', 'recency']]

# instantiate
scaler = StandardScaler()
# fit_transform
rfm_df_scaled = scaler.fit_transform(rfm_df)
rfm_df_scaled.shape

rfm_df_scaled = pd.DataFrame(rfm_df_scaled)
rfm_df_scaled.columns = ['amount', 'frequency', 'recency']
rfm_df_scaled.head()

4. Modelling
# k-means with some arbitrary k
kmeans = KMeans(n_clusters=4, max_iter=50)
kmeans.fit(rfm_df_scaled)
kmeans.labels_
# assign the label
grouped_df['cluster_id'] = kmeans.labels_
grouped_df.head()

# plot
sns.boxplot(x='cluster_id', y='amount', data=grouped_df)

Operating Procedure
```

Open Jupyter note book
Take a new python file
Type the code
Run it
Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting
Precautions:
Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss.
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable.
Readability: Write clean, well-commented code; use markdown for explanations.
Check Dependencies: Ensure all required libraries are installed using !pip list or !conda list.
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.

Troubleshooting:
Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs..
Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.
Browser Issues: Clear browser cache or switch browsers if the UI misbehaves.
Documentation: Refer to Jupyter's official docs and community forums for help.

Observations
Observe the results obtained in each operation.

Calculations & Analysis
Calculations should be given for each operation.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions
What is K means Clustering Algorithm?
Is Feature Scaling required for the K means Algorithm?
Why do you prefer Euclidean distance over Manhattan distance in the K means Algorithm?
Does centroid initialization affect K means Algorithm?
What are the advantages and disadvantages of the K means Algorithm?
How to decide the optimal number of K in the K means Algorithm?
What are the possible stopping conditions in the K means Algorithm?
What is the effect of the number of variables on the K means Algorithm?

Extension and Follow-up Activities (if applicable)

NA

Assessments

Suggested reading

NA

Experiment No 7 (To familiarize the students with data visualization using one feature variables)

Aim/Purpose of the Experiment

To familiarize the students with data visualization using one feature variables.

Learning Outcomes
Knowledge of the Data cleaning, Data preparation and data visualization using univariate analysis in python.

Prerequisites

Basic knowledge of programming, python syntax, matplotlib, seaborn, different libraries.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory
Univariate analysis is a statistical method used to describe and understand the distribution, central tendency, and variability of a single variable. In Python, you can perform univariate analysis using libraries such as NumPy, Pandas, and Matplotlib/Seaborn for data manipulation, analysis, and visualization. Here's a brief outline of the process:

Data Preparation: Load your dataset into a Pandas DataFrame and clean/preprocess the data if necessary. Ensure that the variable of interest is properly formatted and ready for analysis.

Descriptive Statistics: Compute descriptive statistics for the variable of interest. This includes measures such as mean, median, mode, standard deviation, variance, minimum, maximum, and quartiles. These statistics provide an initial understanding of the distribution and characteristics of the variable.

Visualization: Create visualizations to explore the distribution of the variable. Common plots for univariate analysis include histograms, box plots, density plots, and bar plots. Matplotlib and Seaborn are popular libraries for creating these visualizations.

Central Tendency: Analyze the central tendency of the variable by examining the mean, median, and mode. This helps to understand the typical or central value around which the data is distributed.

Variability: Explore the variability of the variable using measures such as standard deviation and variance. Variability indicates how spread out the data points are from the central tendency and provides insights into the dispersion of the data.

Skewness and Kurtosis: Examine skewness and kurtosis to understand the shape of the distribution. Skewness measures the asymmetry of the distribution, while kurtosis measures the tail heaviness or peakedness of the distribution.

Outlier Detection: Identify outliers in the data, which are data points that significantly deviate from the rest of the

observations. Outliers can be detected visually using box plots or statistically using methods such as z-scores or interquartile range (IQR).

Interpretation: Interpret the results of the analysis in the context of your research or problem domain. Draw conclusions about the distribution, central tendency, and variability of the variable, and consider any implications for further analysis or decision-making.
Univariate Analysis

```
import pandas as pd
import numpy as np
df=pd.read_csv('iris.csv')
df
#Finding the data types of variables in the DataFrame
df.dtypes

#Importing libraries essential for data visualization
#MATPLOTLIB
import matplotlib.pyplot as plt
%matplotlib i
#SEABORN
import seaborn as sns
#Plots for continuous variables' analysis
#ENUMERATIVE PLOTS
#UNIVARIATE SCATTER PLOT
plt.scatter(df.index,df['sepal.width'])
plt.show()
sns.scatterplot(x=df.index,y=df['sepal.width'],hue=df['variety'])
#LINE PLOT WITH MARKERS
#Setting title, figure size, labels and font size in matplotlib
plt.figure(figsize=(6,6))
plt.title('Line plot of petal length')
plt.xlabel('index',fontsize=20)
plt.ylabel('petal length',fontsize=20)
plt.plot(df.index,df['petal.length'],markevery=1,marker='d')
for name, group in df.groupby('variety'):
    plt.plot(group.index, group['petal.length'], label=name,markevery=1,marker='d')
plt.legend()
plt.show()
#Setting title, figure size,labels and font size in seaborn
sns.set(rc={'figure.figsize':(7,7)})
sns.set(font_scale=1.5)
fig=sns.lineplot(x=df.index,y=df['petal.length'],markevery=1,marker='d',data=df,hue=df['variety'])
fig.set(xlabel='index')
#STRIP PLOT
sns.stripplot(y=df['sepal.width'])
# Strip-plot(category wise)
```

```
sns.stripplot(x=df['variety'],y=df['sepal.width'])
#SWARM PLOT
#Setting figure size
sns.set(rc={'figure.figsize':(5,5)})
#Swarm-plot
sns.swarmplot(x=df['sepal.width'])
#Swarm-plot category wise
sns.swarmplot(x=df['variety'],y=df['sepal.width'])
#SUMMARY PLOTS
#HISTOGRAM
plt.hist(df['petal.width'])
sns.distplot(df['petal.width'],kde=False,color='black',bins=10)

#DENSITY PLOT
plt.figure(figsize=(5,5))
df['petal.length'].plot(kind='density')
sns.set(rc={'figure.figsize':(5,5)})
sns.kdeplot(df['petal.length'],shade=True)
#RUG PLOT
fig, ax = plt.subplots()
sns.rugplot(df['sepal.length'])
ax.set_xlim(3,9)
plt.show()
from scipy import stats
import numpy as np
kdf=df['sepal.length'].to_numpy()
rdf=np.hstack(kdf)
density = stats.kde.gaussian_kde(rdf)
x = np.arange(3,9,0.1)
plt.plot(x, density(x))
plt.plot(rdf,[0.01]*len(rdf), '|')
sns.distplot(df['sepal.length'],rug=True,hist=False)
#BOX PLOT
plt.boxplot(df['sepal.width'])
#Removing the column with categorical variables
dfM=df.drop('variety',axis=1)
plt.figure(figsize=(9,9))
#Set Title
plt.title('Box plots of the 4 variables')
plt.boxplot(dfM.values,labels=['SepalLength','SepalWidth','PetalLength','PetalWidth'])
sns.boxplot(df['sepal.width'])
sns.set(rc={'figure.figsize':(9,9)})
sns.boxplot(x="variable", y="value", data=pd.melt(dfM))
#distplot()
sns.set(rc={'figure.figsize':(6,6)})
sns.distplot(df['petal.length'],color='black',rug=True)
```

```
#VIOLIN PLOT
plt.figure(figsize=(7,7))
plt.violinplot(dfM.values,showmedians=True)
sns.set(rc={'figure.figsize':(5,5)})
sns.violinplot(df['sepal.width'],orient='vertical')
sns.set(rc={'figure.figsize':(9,9)})
sns.violinplot(x=df['variety'], y=df['petal.width'],data=df)
#Plots for categorical variables' analysis
#BAR PLOT
df['variety'].value_counts().plot.bar()
sns.countplot(df['variety'])
#PIE CHART
plt.pie(df['variety'].value_counts(),labels=['SETOSA','VERSICOLOR','VIRGINICA'],shadow=True)
df1=df.sample(frac=0.35)
plt.figure(figsize=(5,5))
plt.pie(df1['variety'].value_counts(),startangle=90,autopct='%.3f',labels=['SETOSA','VERSICOLOR','VIRGINICA'],shadow=True)
```

Operating Procedure

Open Jupyter note book
Take a new python file
Type the code
Run it
Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting

Precautions:

Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss.
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable..
Readability: Write clean, well-commented code; use markdown for explanations.
Check Dependencies: Ensure all required libraries are installed using !pip list or !conda list..
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.

Troubleshooting:

Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs.
Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.
Browser Issues: Clear browser cache or switch browsers if the UI misbehaves.
Documentation: Refer to Jupyter's official docs and community forums for help.

Observations

Observe the results obtained in each operation.

Calculations & Analysis

Calculations should be given for each operation.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions

You need to check the relationship between the two variables. Which graph would you use?
You need to check if a variable has outliers. Which graph would you use?
You need to perform a univariate analysis. Which graph will you use?
What is a data cleaning step?
What are the ways to handle missing data?
What are some of the methods for univariate analysis?
What problems can outliers cause?
Extension and Follow-up Activities (if applicable)
NA

Assessments

Suggested reading

Experiment No 8 (To familiarize the students with data visualization using two feature variables)

Aim/Purpose of the Experiment

To familiarize the students with data visualization using two feature variables.

Learning Outcomes
Knowledge of the Data cleaning, Data preparation and data visualization using bivariate analysis in python.

Prerequisites

Basic knowledge of programming, python syntax, matplotlib, seaborn, different libraries.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory

Bivariate analysis is a statistical method used to examine the relationship between two variables. In Python, you can perform bivariate analysis using libraries such as NumPy, Pandas, and Matplotlib/Seaborn for data manipulation, analysis, and visualization. Here's a brief outline of the process:

Data Preparation: Load your dataset into a Pandas DataFrame and clean/preprocess the data if necessary. Ensure that the two variables of interest are numeric or can be appropriately converted into numeric format.

Descriptive Analysis: Compute descriptive statistics for each variable separately using methods like mean, median, standard deviation, etc. This provides initial insights into the characteristics of the variables.

Visualization: Create visualizations to explore the relationship between the two variables. Common plots for bivariate analysis include scatter plots, line plots, box plots, and correlation matrices. Seaborn is particularly useful for creating attractive statistical visualizations.

Correlation Analysis: Calculate the correlation coefficient between the two variables to measure the strength and direction of the linear relationship. Pearson correlation coefficient is commonly used for this purpose.
Case Study:
Term deposits also called fixed deposits, are the cash investments made for a specific time period ranging from 1 month to 5 years for predetermined fixed interest rates. The fixed interest rates offered for term deposits are higher than the regular interest rates for savings accounts.
The customers receive the total amount (investment plus the interest) at the end of the maturity period. Also, the money can only be withdrawn at the end of the maturity period. Withdrawing money before that will result in an added penalty associated, and the customer will not receive any interest returns.
Your target is to do end to end EDA on this bank telemarketing campaign data set to infer knowledge that where bank has to put more effort to improve it's positive response rate.

Bivariate Analysis

```
#import the warnings.
import warnings
warnings.filterwarnings("ignore")
#import the useful libraries.
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns
%matplotlib inline

#read the file in inp0 without first two rows as it is of no use.
inp0=pd.read_csv("bank_marketing_updated_v1.csv", skiprows= 2)
#print the head of the data frame.
inp0.head()
#drop the customer id as it is of no use.
inp0.drop("customerid", axis=1, inplace=True)
inp0.head()

#Extract job in newly created 'job' column from "jobedu" column.
inp0['job']=inp0.jobedu.apply(lambda x: x.split(",")[0])
inp0.head()
#Extract education in newly created 'education' column from "jobedu"
column.
inp0['education']=inp0.jobedu.apply(lambda x: x.split(",")[1])
inp0.head()
#drop the "jobedu" column from the dataframe.
inp0.drop('jobedu',axis= 1, inplace= True)
inp0.head()
inp0[inp0.month.apply(lambda x: isinstance(x,float))== True]
inp0.isnull().sum()
#count the missing values in age column.
inp0.age.isnull().sum()
#pring the shape of dataframe inp0
inp0.shape
#calculate the percentage of missing values in age column.
float(100.0*20/45211)

#drop the records with age missing in inp0 and copy in inp1 dataframe.
inp1=inp0[-inp0.age.isnull()].copy()
inp1.shape
#count the missing values in month column in inp1.
inp1.month.isnull().sum()
#print the percentage of each month in the data frame inp1.
float(100.0*50/45191)
#find the mode of month in inp1
month_mode=inp1.month.mode()[0]
```

```
month_mode
# fill the missing values with mode value of month in inp1.
inp1.month.fillna(month_mode, inplace= True)
inp1.month.value_counts(normalize= True)
#let's see the null values in the month column.
inp1.month.isnull().sum()
#count the missing values in response column in inp1.
inp1.response.isnull().sum()
#calculate the percentage of missing values in response column.
float(100.0*30/45191)
#drop the records with response missings in inp1.
inp1= inp1[~inp1.response.isnull()]
#calculate the missing values in each column of data frame: inp1.
inp1.isnull().sum()
#describe the pdays column of inp1.
inp1.pdays.describe()

#describe the pdays column with considering the -1 values.
inp1.loc[inp1.pdays<0,"pdays"]=np.NaN
inp1.pdays.describe()
#plot the scatter plot of balance and salary variable in inp1
plt.scatter(inp1.salary, inp1.balance)
plt.show()
#plot the scatter plot of balance and age variable in inp1
inp1.plot.scatter(x='age', y='balance')
plt.show()
#plot the pair plot of salary, balance and age in inp1 dataframe.
sns.pairplot(data=inp1, vars=["salary","balance", "age"])
plt.show()
#plot the correlation matrix of salary, balance and age in inp1
dataframe.
sns.heatmap( inp1[["salary","balance", "age"]].corr(), annot= True,
cmap= "Reds")
plt.show()

#groupby the response to find the mean of the salary with response no
& yes seperatly.
inp1.groupby("response")["salary"].mean()
#groupby the response to find the median of the salary with response
no & yes seperatly.
inp1.groupby("response")["salary"].median()
#plot the box plot of salary for yes & no responses.
sns.boxplot(data=inp1,x="response", y="salary")
plt.show()
#plot the box plot of balance for yes & no responses.
sns.boxplot(data=inp1,x="response", y="balance")
```

```
plt.show()
#groupby the response to find the mean of the balance with response no
& yes seperatly.
inp1.groupby("response")["balance"].mean()
#groupby the response to find the median of the balance with response
no & yes seperatly.
inp1.groupby("response")["balance"].median()
#function to find the 75th percentile.
def p75(x):
 return np.quantile(x, 0.75)
#calculate the mean, median and 75th percentile of balance with
response
inp1.groupby("response")["balance"].aggregate(["mean","median",p75])
#plot the bar graph of balance's mean an median with response.
inp1.groupby("response")
["balance"].aggregate(["mean","median"]).plot.bar()
plt.show()

#groupby the education to find the mean of the salary education
category.
inp1.groupby("education")["salary"].mean()
#groupby the education to find the median of the salary for each
education category.
inp1.groupby("education")["salary"].median()
#groupby the job to find the mean of the salary for each job category.
inp1.groupby('job')['salary'].mean()
inp1.groupby('job')['salary'].median()

inp1["response_flag"]=np.where(inp1.response=="yes", 1, 0)
inp1.response.value_counts()

inp1.response.value_counts(normalize= True)
inp1.response_flag.mean()
inp1.groupby("education")["response_flag"].mean()
inp1.groupby(["marital"])["response_flag"].mean()
inp1.groupby(["marital"])["response_flag"].mean().plot.barh()
plt.show()

#plot the bar graph of personal loan status with average value of
response_flag
inp1.groupby(["loan"])["response_flag"].mean().plot.bar()
plt.show()

#plot the bar graph of housing loan status with average value of
response_flag
inp1.groupby(["housing"])["response_flag"].mean().plot.bar()
```

```
plt.show()

#plot the boxplot of age with response_flag
sns.boxplot(data=inp1, x="response",y="age")
plt.show()

#plot the bar graph of job categories with response_flag mean value.
inp1.groupby(['job'])['response_flag'].mean().plot.barh()
plt.show()
```

Operating Procedure
Open Jupyter note book
Take a new python file
Type the code
Run it
Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting
Precautions:
Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss.
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable..
Readability: Write clean, well-commented code; use markdown for explanations.
Check Dependencies: Ensure all required libraries are installed using !pip list or !conda list..
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.

Troubleshooting:
Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs..
Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.
Browser Issues: Clear browser cache or switch browsers if the UI misbehaves..
Documentation: Refer to Jupyter's official docs and community forums for help..

Observations
Observe the results obtained in each operation.

Calculations & Analysis
Calculations should be given for each operation.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions

You need to check the relationship between the two variables. Which graph would you use?
You need to check if a variable has outliers. Which graph would you use?
You need to perform a univariate analysis. Which graph will you use?
What is a data cleaning step?
What are the ways to handle missing data?
What are some of the methods for univariate analysis?
What problems can outliers cause?

Extension and Follow-up Activities (if applicable)

NA

Assessments

Suggested reading

NA

Experiment No 9 (To familiarize the students with data visualization using more than two feature variables)

Aim/Purpose of the Experiment

To familiarize the students with data visualization using more than two feature variables.

Learning Outcomes
Knowledge of the Data cleaning, Data preparation and data visualization using multivariate analysis in python.

Prerequisites

Basic knowledge of programming, python syntax, matplotlib, seaborn, different libraries.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory

Multivariate analysis is a statistical method used to understand the relationship between multiple variables simultaneously. In Python, you can perform multivariate analysis using libraries such as NumPy, Pandas, and Matplotlib/Seaborn for data manipulation, analysis, and visualization. Here's a brief outline of the process:

Data Preparation: Load your dataset into a Pandas DataFrame and clean/preprocess the data if necessary. Ensure that the variables of interest are properly formatted and ready for analysis.

Descriptive Statistics: Compute descriptive statistics for all variables in the dataset. This includes measures such as mean, median, mode, standard deviation, variance, minimum, maximum, and quartiles for each variable.

Visualization: Create visualizations to explore the relationships between multiple variables. Common plots for multivariate analysis include scatter plots, pair plots, heatmap correlation matrices, and parallel coordinate plots. Matplotlib and Seaborn are popular libraries for creating these visualizations.

Correlation Analysis: Calculate correlation coefficients between pairs of variables to measure the strength and direction of the linear relationship. This helps identify potential associations between variables and can guide further analysis.

Dimensionality Reduction: If dealing with a high-dimensional dataset, consider dimensionality reduction techniques such as Principal Component Analysis (PCA) or t-distributed Stochastic Neighbor Embedding (t-SNE) to visualize and analyze the data in a lower-dimensional space while preserving its important features.

Cluster Analysis: Explore patterns and groupings within the data using clustering techniques such as K-means clustering or hierarchical clustering. This helps identify naturally occurring clusters of similar observations based on the values of multiple variables.

Regression Analysis: Perform regression analysis to model the relationship between one or more independent variables and a dependent variable. This can help predict the value of the dependent variable based on the values of the independent variables.

Hypothesis Testing (Optional): If applicable, conduct hypothesis tests to determine if there are significant differences or associations between groups or variables. This could involve techniques such as ANOVA, Chi-square tests, or regression analysis with hypothesis tests on coefficients.

Interpretation: Interpret the results of the analysis in the context of your research or problem domain. Draw conclusions about the relationships, patterns, and associations observed between multiple variables and consider any implications for further analysis or decision-making.
Case Study:
Term deposits also called fixed deposits, are the cash investments made for a specific time period ranging from 1 month to 5 years for predetermined fixed interest rates. The fixed interest rates offered for term deposits are higher than the regular interest rates for savings accounts. The customers receive the total amount (investment plus the interest) at the end of the maturity period. Also, the money can only be withdrawn at the end of the maturity period. Withdrawing money before that will result in an added penalty associated, and the customer will not receive any interest returns.
Your target is to do end to end EDA on this bank telemarketing campaign data set to infer knowledge that where bank has to put more effort to improve it's positive response rate.

Multivariate Analysis

```
#import the warnings.
import warnings
warnings.filterwarnings("ignore")
#import the useful libraries.
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns
%matplotlib inline

#read the file in inp0 without first two rows as it is of no use.
inp0=pd.read_csv("bank_marketing_updated_v1.csv", skiprows= 2)
#print the head of the data frame.
inp0.head()

#drop the customer id as it is of no use.
inp0.drop("customerid", axis=1, inplace=True)
inp0.head()

#Extract job in newly created 'job' column from "jobedu" column.
inp0['job']=inp0.jobedu.apply(lambda x: x.split(",")[0])
inp0.head()

#Extract education in newly created 'education' column from "jobedu"
```

column.

```
inp0['education']=inp0.jobedu.apply(lambda x: x.split(",")[1])
inp0.head()

#drop the "jobedu" column from the dataframe.
inp0.drop('jobedu',axis= 1, inplace= True)
inp0.head()

inp0[inp0.month.apply(lambda x: isinstance(x,float))== True]

inp0.isnull().sum()

#count the missing values in age column.
inp0.age.isnull().sum()

#pring the shape of dataframe inp0
inp0.shape

#calculate the percentage of missing values in age column.
float(100.0*20/45211)

#drop the records with age missing in inp0 and copy in inp1 dataframe.
inp1=inp0[-inp0.age.isnull()].copy()
inp1.shape

#count the missing values in month column in inp1.
inp1.month.isnull().sum()

#print the percentage of each month in the data frame inp1.
float(100.0*50/45191)

#find the mode of month in inp1
month_mode=inp1.month.mode()[0]
month_mode

# fill the missing values with mode value of month in inp1.
inp1.month.fillna(month_mode, inplace= True)
inp1.month.value_counts(normalize= True)


#let's see the null values in the month column.
inp1.month.isnull().sum()

#count the missing values in response column in inp1.
inp1.response.isnull().sum()
```

```
#calculate the percentage of missing values in response column.
float(100.0*30/45191)

#drop the records with response missings in inp1.
inp1= inp1[~inp1.response.isnull()]
#calculate the missing values in each column of data frame: inp1.
inp1.isnull().sum()

#describe the pdays column of inp1.
inp1.pdays.describe()

#describe the pdays column with considering the -1 values.
inp1.loc[inp1.pdays<0,"pdays"]=np.NaN
inp1.pdays.describe()

res=pd.pivot_table(data=inp1, index="education", columns="marital",
values="response_flag")
res

#create heat map of education vs marital vs response_flag
sns.heatmap(res, annot= True, cmap="RdYlGn")
plt.show()

sns.heatmap(res, annot= True, cmap="RdYlGn", center= 0.117)
plt.show()

res=pd.pivot_table(data=inp1, index="job", columns="marital",
values="response_flag")
res

sns.heatmap(res, annot= True, cmap="RdYlGn", center= 0.117)
plt.show()

#create the heat map of education vs poutcome vs response_flag.
res=pd.pivot_table(data=inp1, index="education", columns="poutcome",
values="response_flag")

sns.heatmap(res, annot= True, cmap="RdYlGn", center= 0.117)
plt.show()

inp1[inp1.pdays>0].response_flag.mean()

res=pd.pivot_table(data=inp1, index="education", columns="poutcome",
values="response_flag")
sns.heatmap(res, annot= True, cmap="RdYlGn", center= 0.2308)
plt.show()
```

Operating Procedure
Open Jupyter note book
Take a new python file
Type the code
Run it
Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting
Precautions:
Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss.
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable..
Readability: Write clean, well-commented code; use markdown for explanations.
heck Dependencies: Ensure all required libraries are installed using !pip list or !conda list..
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.

Troubleshooting:
Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs.
Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.
Browser Issues: Clear browser cache or switch browsers if the UI misbehaves.
Documentation: Refer to Jupyter's official docs and community forums for help.

Observations
Observe the results obtained in each operation.

Calculations & Analysis
Calculations should be given for each operation.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions
You need to check the relationship between the two variables. Which graph would you use?
You need to check if a variable has outliers. Which graph would you use?
You need to perform a univariate analysis. Which graph will you use?
What is a data cleaning step?
What are the ways to handle missing data?
What are some of the methods for univariate analysis?

What problems can outliers cause?

Extension and Follow-up Activities (if applicable)

NA

Assessments

Suggested reading

NA

Experiment No 10 (To familiarize the students with model evaluation methods using R square, Adjusted R square and VIF for a linear regression)

Aim/Purpose of the Experiment

To familiarize the students with model evaluation methods using R square, Adjusted R square and VIF for a linear regression.

Learning Outcomes

Knowledge of the model evaluation methods using R square, Adjusted R square and VIF for a linear regression.in python.

Prerequisites

Basic knowledge of programming, python syntax, matplotlib, seaborn, different libraries.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory

R2 (R-squared):
R^2 is a measure of how well the independent variables explain the variability of the dependent variable in the model.
It ranges from 0 to 1, with higher values indicating a better fit.
However, R^2 tends to increase as you add more independent variables to the model, even if they don't improve the model's predictive power.
Therefore, it's essential to use adjusted R^2 when comparing models with different numbers of predictors.
R-squared = (TSS-RSS)/TSS
TSS = Total variation in target variable is the sum of squares of the difference between the actual values and their mean.
RSS = Sum of square of the Residual obtained for a point where Residual in the data is the difference between the actual value and the value predicted by the linear regression model.
Adjusted R^2 (adjusted R-squared):
Adjusted R^2 takes into account the number of predictors in the model and penalizes overly complex models.
It adjusts R^2 downward if adding more predictors doesn't significantly improve the model's fit.
Like R^2, it ranges from 0 to 1, with higher values indicating a better fit.
Adjusted R^2 is preferred over R^2 for comparing models with different numbers of predictors.

$$Adjusted\ R^2 = \{1 - [\frac{(1 - R^2)(n-1)}{(n-k-1)}]\}$$

Here,

n represents the number of data points in our dataset
k represents the number of independent variables, and
R represents the R-squared values determined by the model.
Variance Inflation Factor (VIF):
VIF measures the extent to which the variance of an estimated regression coefficient increases because of multicollinearity.
Multicollinearity occurs when independent variables in a regression model are highly correlated with each other.
VIF values greater than 10 are often considered indicative of multicollinearity, although the threshold may vary depending on the context.
High VIF values suggest that the regression coefficients may be unstable due to multicollinearity, which can lead to unreliable estimates and inflated standard errors.

$$VIF_i = \frac{1}{1 - R_i^2} = \frac{1}{\text{Tolerance}}$$

To evaluate a linear regression model using these metrics in Python, you can follow these steps:
Fit the linear regression model using a library such as scikit-learn or statsmodels.
Calculate ₹2R2 and adjusted ₹2R2 to assess the goodness of fit.
Calculate VIF for each predictor variable to check for multicollinearity.

Step 1: Reading and Understanding the Data
Let us first import NumPy and Pandas and read the housing dataset

```
# Supress Warnings
import warnings
warnings.filterwarnings('ignore')
import numpy as np
import pandas as pd
housing = pd.read_csv("Housing.csv")
# Check the head of the dataset
housing.head()

housing.shape

housing.info()

housing.describe()
sns.pairplot(housing)
plt.show()
plt.figure(figsize=(20, 12))
plt.subplot(2,3,1)
sns.boxplot(x = 'mainroad', y = 'price', data = housing)
plt.subplot(2,3,2)
sns.boxplot(x = 'guestroom', y = 'price', data = housing)
```

```
plt.subplot(2,3,3)
sns.boxplot(x = 'basement', y = 'price', data = housing)
plt.subplot(2,3,4)
sns.boxplot(x = 'hotwaterheating', y = 'price', data = housing)
plt.subplot(2,3,5)
sns.boxplot(x = 'airconditioning', y = 'price', data = housing)
plt.subplot(2,3,6)
sns.boxplot(x = 'furnishingstatus', y = 'price', data = housing)
plt.show()

plt.figure(figsize = (10, 5))
sns.boxplot(x = 'furnishingstatus', y = 'price', hue =
'airconditioning', data = housing)
plt.show()
varlist = ['mainroad', 'guestroom', 'basement', 'hotwaterheating',
'airconditioning', 'prefarea']
# Defining the map function
def binary_map(x):
 return x.map({'yes': 1, "no": 0})
# Applying the function to the housing list
housing[varlist] = housing[varlist].apply(binary_map)
# Check the housing dataframe now
housing.head()

# Get the dummy variables for the feature 'furnishingstatus' and store
it in a new variable - 'status'
status = pd.get_dummies(housing['furnishingstatus'])
# Check what the dataset 'status' looks like
status.head()

# Let's drop the first column from status df using 'drop_first = True'
status = pd.get_dummies(housing['furnishingstatus'], drop_first =
True)
# Add the results to the original housing dataframe
housing = pd.concat([housing, status], axis = 1)
# Now let's see the head of our dataframe.
housing.head()


# Drop 'furnishingstatus' as we have created the dummies for it
housing.drop(['furnishingstatus'], axis = 1, inplace = True)
housing.head()

from sklearn.model_selection import train_test_split
# We specify this so that the train and test data set always have the
same rows, respectively
```

```
np.random.seed(0)
df_train, df_test = train_test_split(housing, train_size = 0.7,
test_size = 0.3, random_state = 100)

from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
# Apply scaler() to all the columns except the 'yes-no' and 'dummy'
variables
num_vars = ['area', 'bedrooms', 'bathrooms', 'stories',
'parking','price']
df_train[num_vars] = scaler.fit_transform(df_train[num_vars])
df_train.head()

# Let's check the correlation coefficients to see which variables are
highly correlated
plt.figure(figsize = (16, 10))
sns.heatmap(df_train.corr(), annot = True, cmap="YlGnBu")
plt.show()
```

As you might have noticed, area seems to the correlated to price the most. Let's see a pairplot for area vs price.

```
plt.figure(figsize=[6,6])
plt.scatter(df_train.area, df_train.price)
plt.show()

y_train = df_train.pop('price')
X_train = df_train
import statsmodels.api as sm
# Add a constant
X_train_lm = sm.add_constant(X_train[['area']])
# Create a first fitted model
lr = sm.OLS(y_train, X_train_lm).fit()
# Check the parameters obtained
lr.params

# Let's visualise the data with a scatter plot and the fitted
regression line
plt.scatter(X_train_lm.iloc[:, 1], y_train)
plt.plot(X_train_lm.iloc[:, 1], 0.127 + 0.462*X_train_lm.iloc[:, 1], 'r')
plt.show()
# Print a summary of the linear regression model obtained
print(lr.summary())
# Assign all the feature variables to X
X_train_lm = X_train[['area', 'bathrooms']]
# Build a linear model
import statsmodels.api as sm
```

```
X_train_lm = sm.add_constant(X_train_lm)
lr = sm.OLS(y_train, X_train_lm).fit()
lr.params

# Check the summary
print(lr.summary())

X_train_lm = X_train[['area', 'bathrooms','bedrooms']]
# Build a linear model
import statsmodels.api as sm
X_train_lm = sm.add_constant(X_train_lm)
lr = sm.OLS(y_train, X_train_lm).fit()
lr.params

# Print the summary of the model
print(lr.summary())

Adding all the variables to the model
# Check all the columns of the dataframe
housing.columns
Index(['price', 'area', 'bedrooms', 'bathrooms', 'stories', 'mainroad', 'guestroom', 'basement', 'hotwaterheating',
'airconditioning', 'parking', 'prefarea', 'semi-furnished', 'unfurnished'], dtype='object')
#Build a linear model
import statsmodels.api as sm
X_train_lm = sm.add_constant(X_train)
lr_1 = sm.OLS(y_train, X_train_lm).fit()
lr_1.params

print(lr_1.summary())

# Check for the VIF values of the feature variables.
from statsmodels.stats.outliers_influence import
variance_inflation_factor
# Create a dataframe that will contain the names of all the feature
variables and their respective VIFs
vif = pd.DataFrame()
vif['Features'] = X_train.columns
vif['VIF'] = [variance_inflation_factor(X_train.values, i) for i in
range(X_train.shape[1])]
vif['VIF'] = round(vif['VIF'], 2)
vif = vif.sort_values(by = "VIF", ascending = False)
vif
```

Operating Procedure

Open Jupyter note book
Take a new python file
Type the code
Run it

Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting
Precautions:
Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss.
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable..
Readability: Write clean, well-commented code; use markdown for explanations.
Check Dependencies: Ensure all required libraries are installed using !pip list or !conda list..
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.
Troubleshooting:
Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs..
Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.
Browser Issues: Clear browser cache or switch browsers if the UI misbehaves..
Documentation: Refer to Jupyter's official docs and community forums for help..

Observations
Observe the results obtained in each operation.

Calculations & Analysis
Calculations should be given for each operation.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions
You need to check the relationship between the two variables. Which graph would you use?
You need to check if a variable has outliers. Which graph would you use?
You need to perform a univariate analysis. Which graph will you use?
What is a data cleaning step?
What are the ways to handle missing data?
What are some of the methods for univariate analysis?
What problems can outliers cause?
What is R2 square?
What is adjusted R2square?
What is VIF?
Extension and Follow-up Activities (if applicable)
NA
Assessments
Suggested reading
NA

Experiment No 11 (To familiarize the students with model evaluation methods using Confusion Matrix for Logistic Regression)

Aim/Purpose of the Experiment

To familiarize the students with model evaluation methods using Confusion Matrix for Logistic Regression.

Learning Outcomes
Knowledge of the model evaluation methods using Confusion Matrix for Logistic Regression.in python.

Prerequisites

Basic knowledge of programming, python syntax, matplotlib, seaborn, different libraries.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory

Telecom Churn Case Study
With 21 predictor variables we need to predict whether a particular customer will switch to another telecom provider or not. In telecom terminology, this is referred to as churning and not churning, respectively.
Step 1: Importing and Merging Data

```
# Suppressing Warnings
import warnings
warnings.filterwarnings('ignore')
# Importing Pandas and NumPy
import pandas as pd, numpy as np
# Importing all datasets
churn_data = pd.read_csv("churn_data.csv")
churn_data.head()
```

Combining all data files into one consolidated dataframe

```
# Merging on 'customerID'
df_1 = pd.merge(churn_data, customer_data, how='inner',
on='customerID')
# Final dataframe with all predictor variables
telecom = pd.merge(df_1, internet_data, how='inner', on='customerID')
```

Step 2: Inspecting the Dataframe

```
# Let's see the head of our master dataset
telecom.head()
```

```
# Let's check the dimensions of the dataframe
telecom.shape

# let's look at the statistical aspects of the dataframe
telecom.describe()


# Let's see the type of each column
telecom.info()
```

Step 3: Data Preparation
Converting some binary variables (Yes/No) to 0/1

```
# List of variables to map
varlist = ['PhoneService', 'PaperlessBilling', 'Churn', 'Partner',
'Dependents']
# Defining the map function
def binary_map(x):
 return x.map({'Yes': 1, "No": 0})
# Applying the function to the housing list
telecom[varlist] = telecom[varlist].apply(binary_map)
telecom.head()
```

For categorical variables with multiple levels, create dummy features (one-hot encoded)

```
# Creating a dummy variable for some of the categorical variables and
dropping the first one.
dummy1 = pd.get_dummies(telecom[['Contract', 'PaymentMethod',
'gender', 'InternetService']], drop_first=True)
# Adding the results to the master dataframe
telecom = pd.concat([telecom, dummy1], axis=1)
telecom.head()


# Creating dummy variables for the remaining categorical variables and
dropping the level with big names.
# Creating dummy variables for the variable 'MultipleLines'
ml = pd.get_dummies(telecom['MultipleLines'], prefix='MultipleLines')
# Dropping MultipleLines_No phone service column
ml1 = ml.drop(['MultipleLines_No phone service'], 1)
#Adding the results to the master dataframe
telecom = pd.concat([telecom,ml1], axis=1)
# Creating dummy variables for the variable 'OnlineSecurity'.
os = pd.get_dummies(telecom['OnlineSecurity'],
prefix='OnlineSecurity')
os1 = os.drop(['OnlineSecurity_No internet service'], 1)
```

```
# Adding the results to the master dataframe
telecom = pd.concat([telecom,os1], axis=1)
# Creating dummy variables for the variable 'OnlineBackup'.
ob = pd.get_dummies(telecom['OnlineBackup'], prefix='OnlineBackup')
ob1 = ob.drop(['OnlineBackup_No internet service'], 1)
# Adding the results to the master dataframe
telecom = pd.concat([telecom,ob1], axis=1)
# Creating dummy variables for the variable 'DeviceProtection'.
dp = pd.get_dummies(telecom['DeviceProtection'],
prefix='DeviceProtection')
dp1 = dp.drop(['DeviceProtection_No internet service'], 1)
# Adding the results to the master dataframe
telecom = pd.concat([telecom,dp1], axis=1)
# Creating dummy variables for the variable 'TechSupport'.
ts = pd.get_dummies(telecom['TechSupport'], prefix='TechSupport')
ts1 = ts.drop(['TechSupport_No internet service'], 1)
# Adding the results to the master dataframe
telecom = pd.concat([telecom,ts1], axis=1)
# Creating dummy variables for the variable 'StreamingTV'.
st =pd.get_dummies(telecom['StreamingTV'], prefix='StreamingTV')
st1 = st.drop(['StreamingTV_No internet service'], 1)
# Adding the results to the master dataframe
telecom = pd.concat([telecom,st1], axis=1)
# Creating dummy variables for the variable 'StreamingMovies'.
sm = pd.get_dummies(telecom['StreamingMovies'],
prefix='StreamingMovies')
sm1 = sm.drop(['StreamingMovies_No internet service'], 1)
# Adding the results to the master dataframe
telecom = pd.concat([telecom,sm1], axis=1)
telecom.head()
```

Dropping the repeated variables

```
# We have created dummies for the below variables, so we can drop them
telecom =
telecom.drop(['Contract','PaymentMethod','gender','MultipleLines','Int
ernetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
 'TechSupport', 'StreamingTV', 'StreamingMovies'], 1)
#The varaible was imported as a string we need to convert it to float
telecom['TotalCharges'] =
telecom['TotalCharges'].convert_objects(convert_numeric=True)
telecom.info()
```

Checking for Missing Values and Inputing Them

```
# Adding up the missing values (column-wise)
telecom.isnull().sum()
```

```
# Checking the percentage of missing values
round(100*(telecom.isnull().sum()/len(telecom.index)), 2)

# Removing NaN TotalCharges rows
telecom = telecom[~np.isnan(telecom['TotalCharges'])]
# Checking percentage of missing values after removing the missing
values
round(100*(telecom.isnull().sum()/len(telecom.index)), 2)


Step 4: Test-Train Split
from sklearn.model_selection import train_test_split
# Putting feature variable to X
X = telecom.drop(['Churn','customerID'], axis=1)
X.head()


# Putting response variable to y
y = telecom['Churn']
y.head()

# Splitting the data into train and test
X_train, X_test, y_train, y_test = train_test_split(X, y,
train_size=0.7, test_size=0.3, random_state=100)

Step 5: Feature Scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train[['tenure','MonthlyCharges','TotalCharges']] =
scaler.fit_transform(X_train[['tenure','MonthlyCharges','TotalCharges'
]])
X_train.head()

### Checking the Churn Rate
churn = (sum(telecom['Churn'])/len(telecom['Churn'].index))*100
churn


Step 6: Looking at Correlations
# Importing matplotlib and seaborn
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
# Let's see the correlation matrix
plt.figure(figsize = (20,10)) # Size of the figure
```

```
sns.heatmap(telecom.corr(),annot = True)
plt.show()
```

Dropping highly correlated dummy variables

```
X_test =
X_test.drop(['MultipleLines_No','OnlineSecurity_No','OnlineBackup_No',
'DeviceProtection_No','TechSupport_No',
 'StreamingTV_No','StreamingMovies_No'], 1)
X_train =
X_train.drop(['MultipleLines_No','OnlineSecurity_No','OnlineBackup_No'
,'DeviceProtection_No','TechSupport_No',
 'StreamingTV_No','StreamingMovies_No'], 1)
```

Checking the Correlation Matrix

After dropping highly correlated variables now let's check the correlation matrix again.

```
plt.figure(figsize = (20,10))
sns.heatmap(X_train.corr(),annot = True)
plt.show()
```

Step 7: Model Building

Let's start by splitting our data into a training set and a test set.

Running Your First Training Model

```
import statsmodels.api as sm
# Logistic regression model
logm1 = sm.GLM(y_train,(sm.add_constant(X_train)), family =
sm.families.Binomial())
logm1.fit().summary()
```

Step 8: Feature Selection Using RFE

```
from sklearn.linear_model import LogisticRegression
logreg = LogisticRegression()
from sklearn.feature_selection import RFE
rfe = RFE(logreg, 15) # running RFE with 13 variables as
output
rfe = rfe.fit(X_train, y_train)
rfe.support_

col = X_train.columns[rfe.support_]
X_train.columns[~rfe.support_]
```

Assessing the model with StatsModels

```
X_train_sm = sm.add_constant(X_train[col])
logm2 = sm.GLM(y_train,X_train_sm, family = sm.families.Binomial())
```

```
res = logm2.fit()
res.summary()

# Getting the predicted values on the train set
y_train_pred = res.predict(X_train_sm)
y_train_pred[:10]

y_train_pred = y_train_pred.values.reshape(-1)
y_train_pred[:10]

Creating a dataframe with the actual churn flag and the predicted probabilities
y_train_pred_final = pd.DataFrame({'Churn':y_train.values,
'Churn_Prob':y_train_pred})
y_train_pred_final['CustID'] = y_train.index
y_train_pred_final.head()


Creating new column 'predicted' with 1 if Churn_Prob > 0.5 else 0
y_train_pred_final['predicted'] =
y_train_pred_final.Churn_Prob.map(lambda x: 1 if x > 0.5 else 0)
# Let's see the head
y_train_pred_final.head()

from sklearn import metrics
# Confusion matrix
confusion = metrics.confusion_matrix(y_train_pred_final.Churn,
y_train_pred_final.predicted )
print(confusion)

# Let's check the overall accuracy.
print(metrics.accuracy_score(y_train_pred_final.Churn,
y_train_pred_final.predicted))

# Let's take a look at the confusion matrix again
confusion = metrics.confusion_matrix(y_train_pred_final.Churn,
y_train_pred_final.predicted )
confusion

TP = confusion[1,1] # true positive
TN = confusion[0,0] # true negatives
FP = confusion[0,1] # false positives
FN = confusion[1,0] # false negatives
# Let's see the sensitivity of our logistic regression model
TP / float(TP+FN)

# Let us calculate specificity
```

```
TN / float(TN+FP)


# Calculate false postive rate - predicting churn when customer does
not have churned
print(FP/ float(TN+FP))

# positive predictive value
print (TP / float(TP+FP))


# Negative predictive value
print (TN / float(TN+ FN))
```

Operating Procedure

Open Jupyter note book
Take a new python file
Type the code
Run it
Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting

Precautions:

Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss.
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable..
Readability: Write clean, well-commented code; use markdown for explanations.
Check Dependencies: Ensure all required libraries are installed using !pip list or !conda list..
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.

Troubleshooting:

Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs..
Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.
Browser Issues: Clear browser cache or switch browsers if the UI misbehaves.
Documentation: Refer to Jupyter's official docs and community forums for help.

Observations

Observe the results obtained in each operation.

Calculations & Analysis

Calculations should be given for each operation.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions

What is Confusion matrix?
What is sensitivity?
What is specificity?
What is TPR?
What is FPR?
What is precession ?
What is recall?Extension and Follow-up Activities (if applicable)
NA

Assessments

Suggested reading

NA

Experiment No 12 (To familiarize the students with model evaluation methods using Confusion Matrix for K-Means Clustering algorithm)

Aim/Purpose of the Experiment

To familiarize the students with model evaluation methods using Confusion Matrix for K-Means Clustering algorithm.

Learning Outcomes

Knowledge of the model evaluation methods using Confusion Matrix for K-Means clustering.in python.

Prerequisites

Basic knowledge of programming, python syntax, matplotlib, seaborn, different libraries.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory

Importing Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
import warnings
warnings.filterwarnings('ignore')
```

Importing Data

```
df = pd.read_csv('College_Data.csv')
```

Check the head of the data

```
df.head(3)

df.info()


df.isnull().sum()

df.describe()

EDA
sns.set_style('darkgrid')
sns.lmplot('Room.Board','Grad.Rate',data=df, hue='Private',palette='magma_r',size=6,aspect=1,fit_reg=False)

sns.set_style('darkgrid')
sns.lmplot('Outstate','F.Undergrad',data=df, hue='Private',palette='magma_r',size=6,aspect=1,fit_reg=False)

sns.set_style('dark')
h = sns.FacetGrid(df,hue="Private",palette='coolwarm',size=6,aspect=2)
h = h.map(plt.hist,'Outstate',bins=20,alpha=0.7)

sns.set_style('dark')
g = sns.FacetGrid(df,hue="Private",palette='coolwarm',size=6,aspect=2)
g = g.map(plt.hist,'Grad.Rate',bins=20,alpha=0.7)

df.set_value(95, 'Grad.Rate', 100)


df[df['Grad.Rate'] > 100]


sns.set_style('darkgrid')
g = sns.FacetGrid(df,hue="Private",palette='coolwarm',size=6,aspect=2)
g = g.map(plt.hist,'Grad.Rate',bins=20,alpha=0.7)


df=df.drop(['NameofInstitution',],axis=1)

Import KMeans from SciKit Learn.

from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=2)

kmeans.fit(df.drop('Private',axis=1))

The cluster center vectors for unlabeled data
```

```
means=kmeans.cluster_centers_
print(means)

def converter(cluster):
   if cluster=='Yes':
      return 1
   else:
      return 0
df['Cluster'] = df['Private'].apply(converter)

df.head(3)

df.Private.value_counts()
```

Creating a confusion matrix and classification report to see how well the Kmeans clustering worked without being given any labels.

```
from sklearn.metrics import confusion_matrix,classification_report
print(confusion_matrix(df['Cluster'],kmeans.labels_))
print(classification_report(df['Cluster'],kmeans.labels_))
```

Operating Procedure

Open Jupyter note book
Take a new python file
Type the code
Run it
Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting
Precautions:
Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss.
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable..
Readability: Write clean, well-commented code; use markdown for explanations.
Check Dependencies: Ensure all required libraries are installed using !pip list or !conda list..
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.

Troubleshooting:
Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs..

Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.
Browser Issues: Clear browser cache or switch browsers if the UI misbehaves..
Documentation: Refer to Jupyter's official docs and community forums for help..

Observations
Observe the results obtained in each operation.

Calculations & Analysis
Calculations should be given for each operation.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions
What is K means Clustering Algorithm?
Is Feature Scaling required for the K means Algorithm?
Why do you prefer Euclidean distance over Manhattan distance in the K means Algorithm?
Which metrics can you use to find the accuracy of the K means Algorithm?
What is a centroid point in K means Clustering?
Does centroid initialization affect K means Algorithm?
What are the advantages and disadvantages of the K means Algorithm?
Extension and Follow-up Activities (if applicable)
NA
Assessments
Suggested reading
NA

Experiment No 13 (To Obtain a Model for a real-life dataset in python using Linear Regression)

Aim/Purpose of the Experiment

To Obtain a Model for a real-life dataset in python using Linear Regression.

Learning Outcomes
Knowledge of the model formation, evaluation using Linear Regression.

Prerequisites

Basic knowledge of programming, python syntax, matplotlib, seaborn, different libraries.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory

```
import pandas as pd
import numpy as np
from sklearn import metrics
import matplotlib.pyplot as plt

df = pd.read_csv("RELIANCE.NS.csv")
df.head()

df.dtypes
df['Date']=pd.to_datetime(df.Date)
df.shape

df.drop('Adj Close',axis=1,inplace=True)
df['Volume']=df['Volume'].astype(float)
df.dtypes

df.isnull().sum()
df.isna().any()

df_new = df[np.isfinite(df.select_dtypes(include=[np.number])).all(axis=1)]
df_new.head()

df_new['Open'].plot(figsize=(16,6))

x=df_new[['Open','High','Low','Volume']]
y=df_new['Close']

from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,random_state=0)
```

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import  confusion_matrix,accuracy_score
regressor=LinearRegression()


regressor.fit(x_train,y_train)

print(regressor.coef_)

print(regressor.intercept_)

predicted=regressor.predict(x_test)
print(predicted)

dfr=pd.DataFrame({'Actual':y_test,'Predicted':predicted})

print(dfr)

dfr.sort_values("Actual",ascending=False)
dfr.sort_values("Predicted",ascending=False)

lst=[i for i in range(0,len(dfr.Actual))]
ak=dfr.head(len(dfr.Actual))
aka=ak.sort_values("Actual")

plt.plot(lst,aka)
plt.xlabel("Time")
plt.ylabel("Stock Closing Price")
plt.title("Actual Closing Price")
plt.show()

lst=[i for i in range(0,len(dfr.Actual))]
aak=ak.sort_values("Predicted")
plt.plot(lst,aak,color="Yellow")
plt.xlabel("Time")
plt.ylabel("Stock Closing Price")
plt.title("Predicted Closing Price")
plt.show()


regressor.score(x_test,y_test)
```

Operating Procedure
Open Jupyter note book
Take a new python file

Type the code
Run it
Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting
Precautions:
Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss.
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable..
Readability: Write clean, well-commented code; use markdown for explanations.
Check Dependencies: Ensure all required libraries are installed using !pip list or !conda list..
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.

Troubleshooting:
Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs..
Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.
Browser Issues: Clear browser cache or switch browsers if the UI misbehaves..
Documentation: Refer to Jupyter's official docs and community forums for help..

Observations
Observe the results obtained in each operation.

Calculations & Analysis
Calculations should be given for each operation.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions
Can you explain what linear regression is and how it is used in data analysis?
In the context of a linear regression case study, what is the significance of the slope and intercept terms?
How do you evaluate the goodness of fit of a linear regression model?
What are some assumptions underlying linear regression models? How would you verify if these assumptions hold true for your data?
Can you discuss multicollinearity and its impact on linear regression models? How would you address multicollinearity if it's present in your data?
Describe the process you would follow to build a linear regression model for a given dataset.
How do you interpret the coefficients of a linear regression model?
What is the purpose of residual analysis in linear regression? How do you interpret residuals?

How would you handle outliers in the dataset when performing linear regression analysis?
What are some techniques for feature selection in linear regression? Can you explain the pros and cons of each method?
What is the difference between simple linear regression and multiple linear regression? When would you choose one over the other?
Can you explain the concept of regularization in the context of linear regression? How does it help in model improvement?
How would you deal with heteroscedasticity in a linear regression model?
Can you discuss the difference between R-squared and adjusted R-squared? When would you prefer to use adjusted R-squared?
How do you handle categorical variables in a linear regression model?
Can you explain the concept of overfitting in the context of linear regression? How would you prevent overfitting in your model?
What are some common pitfalls or challenges when interpreting the results of a linear regression model?
How would you validate the performance of your linear regression model? What metrics would you use, and why?
Can you discuss the assumptions of normality in linear regression residuals? How would you assess if these assumptions are met?
In what scenarios might linear regression not be an appropriate modeling technique? What alternative approaches could be considered?

Extension and Follow-up Activities (if applicable)

NA

Assessments

Suggested reading

NA

Experiment No 14 (To Obtain a Model for a real-life dataset (Bike Sharing data in Covid Time) in python using Logistic Regression)

Aim/Purpose of the Experiment

To Obtain a Model for a real-life dataset (Bike Sharing data in Covid Time) in python using Logistic Regression.

Learning Outcomes

Knowledge of the model formation, evaluation using Logistic Regression.

Prerequisites

Basic knowledge of programming, python syntax, matplotlib, seaborn, different libraries.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory

Step 1: Reading and Understanding the Data

```
# Suppressing Warnings
import warnings
warnings.filterwarnings('ignore')
```

```
# Importing Different libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns


# Importing the datasets
cd = pd.read_csv("churn_data.csv")
cd2 = pd.read_csv("customer_data.csv")
id = pd.read_csv("internet_data.csv")

# Checking the head of the datasets
cd.head(10

# Checking the head of the datasets
cd2.head(10)

# Checking the head of the datasets
id.head(10)

# Merging all datasets in one common dataset on 'customerID'
cd3 = pd.merge(cd, cd2, how='inner', on='customerID')

# Merging all datasets in one common dataset on 'customerID'
cd4 = pd.merge(cd3, id, how='inner', on='customerID')

Step 2:understanding the structure of the merged dataset

#checking the head of the dataset
cd4.head(10)


# checking the shape of the dataframe
cd4.shape

# Checking at the statistical info of the dataframe
cd4.describe()


# Checkinge the type of each column
cd4.info()

Step 3: Data Preparation
# Extracting the 'object' data types only
cd5=cd4.select_dtypes(include='object')
```

```
cd5.columns


# Checking the unique value counts for object data types
cd5.nunique()

# Checking the columns which have only 2 unique values
cd6 = cd4[["PhoneService","PaperlessBilling","Churn","gender","Partner","Dependents"]]

cd6.head()

# Convert 2 unique (YES/NO) object columns to 1/0 values.
# Converting Yes to 1 and No to 0
cd4['PhoneService'] = cd4['PhoneService'].map({'Yes': 1, 'No': 0})
cd4['PaperlessBilling'] = cd4['PaperlessBilling'].map({'Yes': 1, 'No': 0})
cd4['Churn'] = cd4['Churn'].map({'Yes': 1, 'No': 0})
cd4['Partner'] = cd4['Partner'].map({'Yes': 1, 'No': 0})
cd4['Dependents'] = cd4['Dependents'].map({'Yes': 1, 'No': 0})

cd7 =
cd5[["Contract","MultipleLines","InternetService","OnlineSecurity","OnlineBackup","DeviceProtection","TechSuppor
t","StreamingTV","StreamingMovies"]]
cd7.head()


# Checking the columns which have only 3 unique values
for column in cd7:
   print(cd5[column].value_counts())

# Creating a dummy variable for the variable 'Contract' and dropping the first one.
cont = pd.get_dummies(cd4['Contract'],prefix='Contract',drop_first=True)
#Adding the results to the master dataframe
cd4 = pd.concat([cd4,cont],axis=1)

# Creating a dummy variable for the variable 'PaymentMethod' and dropping the first one.
pm = pd.get_dummies(cd4['PaymentMethod'],prefix='PaymentMethod',drop_first=True)
#Adding the results to the master dataframe
cd4 = pd.concat([cd4,pm],axis=1)

# Creating a dummy variable for the variable 'gender' and dropping the first one.
gen = pd.get_dummies(cd4['gender'],prefix='gender',drop_first=True)
#Adding the results to the master dataframe
cd4 = pd.concat([cd4,gen],axis=1)

# Creating a dummy variable for the variable 'MultipleLines' and dropping the first one.
ml = pd.get_dummies(cd4['MultipleLines'],prefix='MultipleLines')
```

```
#  dropping MultipleLines_No phone service column
ml1 = ml.drop(['MultipleLines_No phone service'],1)
#Adding the results to the master dataframe
cd4 = pd.concat([cd4,ml1],axis=1)

# Creating a dummy variable for the variable 'InternetService' and dropping the first one.
iser = pd.get_dummies(cd4['InternetService'],prefix='InternetService',drop_first=True)
#Adding the results to the master dataframe
cd4 = pd.concat([cd4,iser],axis=1)

# Creating a dummy variable for the variable 'OnlineSecurity'.
os = pd.get_dummies(cd4['OnlineSecurity'],prefix='OnlineSecurity')
os1= os.drop(['OnlineSecurity_No internet service'],1)
#Adding the results to the master dataframe
cd4 = pd.concat([cd4,os1],axis=1)

# Creating a dummy variable for the variable 'OnlineBackup'.
ob =pd.get_dummies(cd4['OnlineBackup'],prefix='OnlineBackup')
ob1 =ob.drop(['OnlineBackup_No internet service'],1)
#Adding the results to the master dataframe
cd4 = pd.concat([cd4,ob1],axis=1)

# Creating a dummy variable for the variable 'DeviceProtection'.
dp =pd.get_dummies(cd4['DeviceProtection'],prefix='DeviceProtection')
dp1 = dp.drop(['DeviceProtection_No internet service'],1)
#Adding the results to the master dataframe
cd4 = pd.concat([cd4,dp1],axis=1)

# Creating a dummy variable for the variable 'TechSupport'.
ts =pd.get_dummies(cd4['TechSupport'],prefix='TechSupport')
ts1 = ts.drop(['TechSupport_No internet service'],1)
#Adding the results to the master dataframe
cd4 = pd.concat([cd4,ts1],axis=1)

# Creating a dummy variable for the variable 'StreamingTV'.
st =pd.get_dummies(cd4['StreamingTV'],prefix='StreamingTV')
st1 = st.drop(['StreamingTV_No internet service'],1)
#Adding the results to the master dataframe
cd4 = pd.concat([cd4,st1],axis=1)

# Creating a dummy variable for the variable 'StreamingMovies'.
sm =pd.get_dummies(cd4['StreamingMovies'],prefix='StreamingMovies')
sm1 = sm.drop(['StreamingMovies_No internet service'],1)
#Adding the results to the master dataframe
cd4 = pd.concat([cd4,sm1],axis=1)
```

```
# Checking the head of the processed dataset
cd4.head()


# Droping the original columns on which dummies are created
cd4 = cd4.drop(['Contract','PaymentMethod','gender','MultipleLines','InternetService', 'OnlineSecurity',
'OnlineBackup', 'DeviceProtection',
      'TechSupport', 'StreamingTV', 'StreamingMovies'], 1)



# Checking the info of dataset
cd4.info()

# Dropping CustomerID as it is useless for model prediction
cd4 = cd4.drop(['customerID'], axis=1)
cd4.head()

# converting the datatype of 'MonthlyCharges','TotalCharges' from object to float
cd4.MonthlyCharges = cd4.MonthlyCharges.astype(float, errors="ignore")
cd4['TotalCharges'] = pd.to_numeric(cd4['TotalCharges'], errors = 'coerce')


# Checking the head of MonthlyCharges
cd4.MonthlyCharges.head()

# Checking the percentage of missing values
round(100*(cd4.isnull().sum()/len(cd4.index)), 2)

# Checking for outliers in the continuous variables
cd8 = cd4[['tenure','MonthlyCharges','SeniorCitizen','TotalCharges']]
sns.boxplot(data = cd8)
plt.show()

# There is sum NAN values in TotalCharge columns, so it should be dropped
cd4[['TotalCharges']].value_counts()
# Removing NaN TotalCharges rows
cd4 = cd4[~np.isnan(cd4['TotalCharges'])]
```

Step4: Model Building Preprocessing

```
# Importing test_train_split
from sklearn.model_selection import train_test_split

# Putting feature variables to X
```

```
X = cd4.drop(['Churn'], axis=1)

X.head()

# Putting response variable to y
y = cd4['Churn']

y.head()

# Splitting the data into train and test
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, test_size=0.3, random_state=100)


import statsmodels.api as sm

# Logistic regression model
logm1 = sm.GLM(y_train,(sm.add_constant(X_train)), family = sm.families.Binomial())
logm1.fit().summary()

# Let's see the correlation matrix
plt.figure(figsize = (20,10))        # Size of the figure
sns.heatmap(cd4.corr(),annot = True)

## Dropping highly correlated variables.

X_test2 =
X_test.drop(['MultipleLines_No','OnlineSecurity_No','OnlineBackup_No','DeviceProtection_No','TechSupport_No','St
reamingTV_No','StreamingMovies_No'],1)
X_train2 =
X_train.drop(['MultipleLines_No','OnlineSecurity_No','OnlineBackup_No','DeviceProtection_No','TechSupport_No','
StreamingTV_No','StreamingMovies_No'],1)

plt.figure(figsize = (20,10))
sns.heatmap(X_train2.corr(),annot = True)
```

Step5: Model Building Using Logistic Regression

```
# # Logistic regression model 2
logm2 = sm.GLM(y_train,(sm.add_constant(X_train2)), family = sm.families.Binomial())
logm2.fit().summary()

from sklearn.linear_model import LogisticRegression
logreg = LogisticRegression()
from sklearn.feature_selection import RFE
rfe = RFE(logreg, n_features_to_select = 13)          # running RFE with 13 variables as output
```

```
rfe = rfe.fit(X,y)
print(rfe.support_)        # Printing the boolean results
print(rfe.ranking_)        # Printing the ranking

# Variables selected by RFE
col = ['PhoneService', 'PaperlessBilling', 'Contract_One year', 'Contract_Two year',
     'PaymentMethod_Electronic check','MultipleLines_No','InternetService_Fiber optic', 'InternetService_No',
     'OnlineSecurity_Yes','TechSupport_Yes','StreamingMovies_No','tenure','TotalCharges']

# Running the model using the selected variables
from sklearn.linear_model import LogisticRegression
from sklearn import metrics
logsk = LogisticRegression(C=1e9)

logsk.fit(X_train, y_train)


#Comparing the model with StatsModels
logm4 = sm.GLM(y_train,(sm.add_constant(X_train)), family = sm.families.Binomial())
modres = logm4.fit()
logm4.fit().summary()


# Checking the shape of test dataset
X_test[col].shape

### Making Predictions

# Predicted probabilities
y_pred = logsk.predict_proba(X_test)
# Converting y_pred to a dataframe which is an array
y_pred_df = pd.DataFrame(y_pred)
# Converting to column dataframe
y_pred_1 = y_pred_df.iloc[:,[1]]
# Checking the head
y_pred_1.head()


# Converting y_test to dataframe
y_test_df = pd.DataFrame(y_test)
y_test_df.head()

# Putting CustID to index
y_test_df['CustID'] = y_test_df.index
# Removing index for both dataframes to append them side by side
y_pred_1.reset_index(drop=True, inplace=True)
```

```
y_test_df.reset_index(drop=True, inplace=True)
# Appending y_test_df and y_pred_1
y_pred_final = pd.concat([y_test_df,y_pred_1],axis=1)
# Renaming the column
y_pred_final= y_pred_final.rename(columns={ 1 : 'Churn_Prob'})
# Rearranging the columns
y_pred_final = y_pred_final.reindex(['CustID','Churn','Churn_Prob'], axis=1)
# Checking the head of y_pred_final
y_pred_final.head()

# Creating new column 'predicted' with 1 if Churn_Prob>0.5 else 0
y_pred_final['predicted'] = y_pred_final.Churn_Prob.map( lambda x: 1 if x > 0.5 else 0)
# Checking the head
y_pred_final.head()

# Model Evaluation

from sklearn import metrics

# Confusion matrix
confusion = metrics.confusion_matrix( y_pred_final.Churn, y_pred_final.predicted )
confusion

#Checking the overall accuracy
metrics.accuracy_score(y_pred_final.Churn, y_pred_final.predicted)

# Define the ROC
def draw_roc( actual, probs ):
    fpr, tpr, thresholds = metrics.roc_curve( actual, probs,
                                              drop_intermediate = False )
    auc_score = metrics.roc_auc_score( actual, probs )
    plt.figure(figsize=(6, 6))
    plt.plot( fpr, tpr, label='ROC curve (area = %0.2f)' % auc_score )
    plt.plot([0, 1], [0, 1], 'k--')
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate or [1 - True Negative Rate]')
    plt.ylabel('True Positive Rate')
    plt.title('Receiver operating characteristic example')
    plt.legend(loc="lower right")
    plt.show()

#draw_roc(y_pred_final.Churn, y_pred_final.predicted)
"{:2.2f}".format(metrics.roc_auc_score(y_pred_final.Churn, y_pred_final.Churn_Prob))
```

```
# Checking the shape of the dataset
X_train.shape
```

Operating Procedure

Open Jupyter note book
Take a new python file
Type the code
Run it
Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting

Precautions:

Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss.
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable..
Readability: Write clean, well-commented code; use markdown for explanations.
Check Dependencies: Ensure all required libraries are installed using !pip list or !conda list..
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.

Troubleshooting:

Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs..
Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.
Browser Issues: Clear browser cache or switch browsers if the UI misbehaves..
Documentation: Refer to Jupyter's official docs and community forums for help..

Observations

Observe the results obtained in each operation.

Calculations & Analysis

Calculations should be given for each operation.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions

Can you explain what logistic regression is and how it is used in data analysis?
In the context of a logistic regression case study, what is the difference between odds and probability?
How do you interpret the coefficients of a logistic regression model?
What are some common applications of logistic regression in real-world scenarios?
Describe the process you would follow to build a logistic regression model for a given dataset.
What are the key assumptions of logistic regression? How would you verify if these assumptions hold true for your data?
Can you discuss the concept of the logit function in logistic regression?
How do you evaluate the performance of a logistic regression model? What metrics would you use, and why?

What is the purpose of the threshold in logistic regression? How do you choose an appropriate threshold for classification?
Can you explain the difference between binary logistic regression and multinomial logistic regression?
How would you handle imbalanced classes in a logistic regression problem?
What techniques can be used for feature selection in logistic regression? Can you discuss the pros and cons of each method?
How do you deal with multicollinearity in logistic regression?
Can you discuss the concept of regularization in logistic regression? How does it help in model improvement?
What are some common pitfalls or challenges when interpreting the results of a logistic regression model?
How would you handle missing data in a logistic regression analysis?
Can you explain the concept of odds ratio in logistic regression? How is it useful in interpreting the relationship between predictors and the outcome variable?
What are some alternatives to logistic regression for binary classification tasks?
In what scenarios might logistic regression not be an appropriate modeling technique? What alternative approaches could be considered?
Can you discuss the difference between logistic regression and linear regression? When would you choose one over the other?

Extension and Follow-up Activities (if applicable)

NA

Assessments

Suggested reading

NA

Experiment No 15 (To Obtain a Model for a real-life dataset (Bike Sharing data in Covid Time) in python using Logistic Regression)

Aim/Purpose of the Experiment

To Obtain a Model for a real-life dataset (Bike Sharing data in Covid Time) in python using Logistic Regression.

Learning Outcomes

Knowledge of the model formation, evaluation using Logistic Regression.

Prerequisites

Basic knowledge of programming, python syntax, matplotlib, seaborn, different libraries.

Materials/Equipment/Apparatus / Devices/Software required

Jupyter Notebook.

Introduction and Theory

```
# Importing the required libraries
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns
%matplotlib inline

# Reading the csv file and putting it into 'df' object.
```

```
df = pd.read_csv('heart_v2.csv')

df.columns

Index(['age', 'sex', 'BP', 'cholestrol', 'heart disease'], dtype='object')

df.head()


# Putting feature variable to X
X = df.drop('heart disease',axis=1)

# Putting response variable to y
y = df['heart disease']


from sklearn.model_selection import train_test_split


X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, random_state=42)
X_train.shape, X_test.shape
```

Fitting the decision tree with default hyperparameters, apart from max_depth which is 3 so that we can plot and read the tree.

```
from sklearn.tree import DecisionTreeClassifier

!pip install six

# Importing required packages for visualization
from IPython.display import Image
from six import StringIO
from sklearn.tree import export_graphviz
import pydotplus, graphviz


 plotting tree with max_depth=3
dot_data = StringIO()

export_graphviz(dt, out_file=dot_data, filled=True, rounded=True,
                feature_names=X.columns,
                class_names=['No Disease', "Disease"])

graph = pydotplus.graph_from_dot_data(dot_data.getvalue())
Image(graph.create_png())
#Image(graph.create_png(),width=800,height=900)
```

```
#graph.write_pdf("dt_heartdisease.pdf")

y_train_pred = dt.predict(X_train)
y_test_pred = dt.predict(X_test)

from sklearn.metrics import confusion_matrix, accuracy_score

print(accuracy_score(y_train, y_train_pred))
confusion_matrix(y_train, y_train_pred)

print(accuracy_score(y_test, y_test_pred))
confusion_matrix(y_test, y_test_pred)
```

Operating Procedure

Open Jupyter note book
Take a new python file
Type the code
Run it
Take inputs from user
Observe the results
Verify the results manually
Store the note book file

Precautions and/or Troubleshooting

Precautions:

Save Your Work: Use the save icon or Ctrl+S/Cmd+S to prevent data loss
Restart Kernel: Resolve errors or glitches by restarting the kernel to reset the notebook.
Clear Outputs: Remove unnecessary outputs to keep the notebook clean and readable..
Readability: Write clean, well-commented code; use markdown for explanations.
Check Dependencies: Ensure all required libraries are installed using !pip list or !conda list..
Kernel Selection: Confirm the kernel matches the programming language/environment.
Resource Usage: Avoid overloading memory/CPU, especially with large datasets.

Troubleshooting:

Syntax Errors: Ensure proper syntax and indentation in your code.
Variable Scope: Avoid variable name conflicts or scope-related bugs..
Library Installation: Install missing packages using !pip install or !conda install.
Kernel Crashes: Optimize code and manage resource use to avoid kernel overload.
Browser Issues: Clear browser cache or switch browsers if the UI misbehaves..
Documentation: Refer to Jupyter's official docs and community forums for help..

Observations

Observe the results obtained in each operation.

Calculations & Analysis

Calculations should be given for each operation.

Result & Interpretation

Result should be printed and pasted in laboratory copy found from Jupyter note book.

Follow-up Questions

Can you explain what a decision tree is and how it is used in machine learning?
Describe the process of building a decision tree for a given dataset.
What are the advantages and disadvantages of decision trees compared to other machine learning algorithms?
How do decision trees handle both categorical and numerical data?
What criteria are commonly used to split nodes in a decision tree?
Can you explain the concept of entropy and information gain in the context of decision trees?
How do you handle missing values in a dataset when using decision trees?
What is the difference between a classification tree and a regression tree?
How do you prevent overfitting when building a decision tree model?
Can you discuss pruning techniques in decision trees? When is pruning necessary, and how does it improve model performance?
How do you interpret the results of a decision tree model?
What are ensemble methods, and how are they related to decision trees?
Can you discuss the difference between bagging and boosting in the context of decision trees?
What are some common algorithms used for building decision trees?
How would you handle imbalanced classes in a classification problem using decision trees?
What are some strategies for feature selection in decision tree-based models?
Can you explain the concept of Gini impurity and how it is used in decision tree algorithms?
How do decision trees handle nonlinear relationships between features and the target variable?
In what scenarios might decision trees not be an appropriate modeling technique? What alternative approaches could be considered?
Can you discuss the importance of hyperparameter tuning in optimizing a decision tree model?

Extension and Follow-up Activities (if applicable)

NA

Assessments

Suggested reading

NA